
Solving Inverse Problems with Flow-based Models via Model Predictive Control

George Webber^{*1} Alexander Denker^{†*2} Riccardo Barbano³ Andrew J Reader¹

Abstract

Flow-based generative models provide strong unconditional priors for inverse problems, but guiding their dynamics for conditional generation remains challenging. Recent work casts training-free conditional generation in flow models as an optimal control problem; however, solving the resulting trajectory optimisation is computationally and memory intensive, requiring differentiation through the flow dynamics or adjoint solves. We propose MPC-Flow, a model predictive control framework that formulates inverse problem solving with flow-based generative models as a sequence of control sub-problems, enabling practical optimal control-based guidance at inference time. We provide theoretical analysis linking MPC-Flow to the underlying optimal control objective and show how different algorithmic choices yield a spectrum of guidance algorithms, including regimes that avoid backpropagation through the generative model trajectory. We evaluate MPC-Flow on benchmark image restoration tasks, spanning linear and non-linear settings such as in-painting, deblurring, and super-resolution, and demonstrate strong performance and scalability to massive state-of-the-art architectures via training-free guidance of FLUX.2 (32B) in a quantised setting on consumer hardware.

1. Introduction

Continuous normalising flows (Chen et al., 2018), particularly those trained via flow matching (Lipman et al., 2023), have become an established and powerful framework for generating high-dimensional data (Esser et al., 2024; Wang

et al., 2025b). These models define a continuous-time transformation through an ordinary differential equation (ODE) that maps samples from a simple base distribution to complex data distributions. Beyond unconditional generation, there is significant interest in leveraging the learned dynamics of these models to solve inverse problems. Here, the goal is to recover a signal $\mathbf{x} \in \mathbb{R}^d$ from noisy measurements

$$\mathbf{y} = \mathcal{A}(\mathbf{x}) + \epsilon, \quad (1)$$

with $\mathcal{A}$ a (possibly non-linear) forward operator and ϵ denoting additive Gaussian noise, i.e., $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$.

Recent work has explored training-free guidance methods that adapt pre-trained flow models to conditioning constraints at inference time (Kim et al., 2025; Pogle et al., 2024; Zhang et al., 2024), see also Appendix A for a discussion on related work. These approaches incorporate data fidelity terms or task-specific loss functions directly into the generative dynamics, steering the flow towards regions that satisfy the desired constraints without retraining the base model. Such approaches are often motivated by the Bayesian perspective on inverse problems (Stuart, 2010), where the prior $p_{\text{data}}(\mathbf{x})$, given by the pre-trained flow model, is combined with likelihood $p(\mathbf{y}|\mathbf{x})$. However, these heuristic guidance strategies have limitations (Chung et al., 2023; Patel et al., 2024; Barbano et al., 2025). They often lack theoretical guarantees, e.g., regarding consistency to the measurements, and can be computationally unstable.

As an alternative, Liu et al. (2023b); Wang et al. (2025a) recast conditional sampling in continuous-time flow models as a deterministic optimal control problem. While the optimal control perspective provides a principled objective and improved trade-offs between data fidelity and prior consistency, solving the resulting trajectory optimisation requires differentiating through the sampling dynamics, leading to prohibitive computational and memory costs, or costly evaluations of the adjoint equations.

We propose MPC-Flow, a model predictive control (MPC) framework (Garcia et al., 1989; Rawlings et al., 2020) that makes optimal control-based guidance practical at inference time. Rather than optimising the entire trajectory at once, as in classical optimal control, MPC-Flow decomposes the control into a sequence of short-horizon sub-problems. At each time step, a local control problem is solved, the re-

^{*}Equal contribution [†]Work done while at University College London. ¹School of Biomedical Engineering and Imaging Sciences, King’s College London ²Helmholtz Imaging, Deutsches Elektronen-Synchrotron DESY, Germany ³Department of Computer Science, University College London. Correspondence to: Alexander Denker <alexander.denker@desy.de>.

sulting control is applied, and this process is repeated in a receding-horizon fashion. Our proposed framework allows for a control strategy that significantly reduces memory requirements by decoupling control optimisation from the full flow trajectory, while improving robustness through iterative re-planning that corrects errors accumulated at earlier time steps. Moreover, different choices of horizon length, control parameterisation, and terminal cost function give rise to a range of guidance algorithms within this unified framework, including an efficient and principled variant that avoids backpropagation through the generative trajectory.

We validate the proposed framework empirically across a range of controlled and large-scale settings. We first analyse MPC-Flow in controlled settings, including a two-dimensional toy example and a computed tomography task on the OrganCMNIST dataset, enabling detailed ablations and direct comparison with globally optimal control solutions. We then evaluate MPC-Flow on a range of linear and non-linear image restoration inverse problems on the CelebA dataset, and finally demonstrate scalability to modern large-scale rectified flow models.

To the best of our knowledge, MPC-Flow is the first conditional generation method explicitly rooted in optimal control that scales to such settings, enabling training-free guidance under tight memory constraints and outperforming existing scalable approaches, including guidance of FLUX.2 (32B) (Labs, 2025) in a quantised setting on consumer hardware.

2. Background

2.1. Flow-based Models

Flow Matching (FM) (Lipman et al., 2023) is a framework for training continuous normalising flows using a simulation-free objective. The goal is to learn a time-dependent vector field $v_\theta : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$ that induces a probability path p_t transforming a simple base distribution p_0 (e.g., a standard Gaussian) into a data distribution $p_{\text{data}} := p_1$. Sample generation is performed by solving the ODE

$$\frac{d\mathbf{x}(t)}{dt} = v_\theta(\mathbf{x}(t), t), \quad \mathbf{x}(0) \sim p_0. \quad (2)$$

Training proceeds by regressing v_θ toward a ground-truth velocity field u_t that generates the desired probability path p_t , by minimising the flow matching objective

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t, \mathbf{x} \sim p_t} \|v_\theta(\mathbf{x}, t) - u_t(\mathbf{x})\|^2.$$

However, the marginal velocity field u_t is generally intractable to compute, as it depends on the unknown intermediate density p_t and its score for complex high-dimensional data distributions. To address this intractability, FM adopts a conditional formulation. Conditional Flow Matching trains

the model to match a *conditional* vector field $u_t(\mathbf{x} \mid \mathbf{x}_0, \mathbf{x}_1)$, with $(\mathbf{x}_0, \mathbf{x}_1) \sim p_0 p_1$, by minimising

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t, \mathbf{x}_0, \mathbf{x}_1, \mathbf{x}} \|v_\theta(\mathbf{x}, t) - u_t(\mathbf{x} \mid \mathbf{x}_0, \mathbf{x}_1)\|^2, \quad (3)$$

Crucially, it has been shown that $\nabla_\theta \mathcal{L}_{\text{CFM}}(\theta) = \nabla_\theta \mathcal{L}_{\text{FM}}(\theta)$ (Lipman et al., 2023; Tong et al., 2024). In practice, the conditional probability path is often defined via an affine interpolation between a source sample $\mathbf{x}_0 \sim p_0$ and a target data point $\mathbf{x}_1 \sim p_1$. A common choice is the linear interpolation used in rectified flows (Liu, 2022), $\mathbf{x}_t = (1 - t)\mathbf{x}_0 + t\mathbf{x}_1$. This induces a simple conditional flow with velocity $u_t(\mathbf{x} \mid \mathbf{x}_0, \mathbf{x}_1) = \mathbf{x}_1 - \mathbf{x}_0$. Substituting this into (3) yields the simplified training objective

$$\mathcal{L}(\theta) = \mathbb{E}_{t, \mathbf{x}_0, \mathbf{x}_1} \|v_\theta(\mathbf{x}_t, t) - (\mathbf{x}_1 - \mathbf{x}_0)\|^2. \quad (4)$$

At inference time, the learned vector field v_θ is integrated according to (2) to generate samples from the model.

2.2. Optimal Control in Flow Models

Liu et al. (2023b); Wang et al. (2025a) frame conditional generation with pre-trained flow models as a deterministic optimal control problem, formalised as

$$\begin{aligned} \min_{\mathbf{u}} \left\{ \mathcal{J}(\mathbf{u}) = \int_0^1 \|\mathbf{u}(t)\|_2^2 dt + \lambda \Phi(\mathbf{x}(1)) \right\} \\ \text{s.t.} \quad \frac{d\mathbf{x}(t)}{dt} = v_\theta(\mathbf{x}(t), t) + \mathbf{u}(t), \quad t \in [0, 1]. \end{aligned} \quad (5)$$

The trajectories are initialised at $\mathbf{x}(0) = \mathbf{x}_0$, sampled from the base distribution p_0 . Here, $v_\theta(\mathbf{x}, t)$ denotes the pre-trained flow model, while the integral term regularises the control energy, encouraging trajectories to remain close to the nominal flow dynamics. The terminal loss $\Phi(\mathbf{x})$ encodes the conditioning objective. This optimisation problem seeks a control $\mathbf{u} : [0, 1] \rightarrow \mathbb{R}^d$ that steers the trajectory to minimise this terminal loss while maintaining minimal deviation from the pre-trained base model.

Under some standard conditions on the flow field v_θ and the terminal cost Φ , we can prove the existence of an optimal control. The proof follows standard arguments and is provided in Appendix B.1.

Remark 2.1. For inverse problems, the terminal loss can be chosen as

$$\Phi(\mathbf{x}) = \frac{1}{2\sigma^2} \|\mathcal{A}(\mathbf{x}) - \mathbf{y}\|_2^2,$$

which corresponds to assuming additive Gaussian noise with variance σ^2 in the likelihood model; see (1). Under this choice, the resulting optimal control formulation admits an interpretation within the framework of variational regularisation (Engl et al., 1996). In particular, the induced regulariser is given by a distance induced by the flow model, measuring how costly it is to reach $\mathbf{x}$ from the initial point $\mathbf{x}_0$. We refer to Appendix H for a detailed discussion.

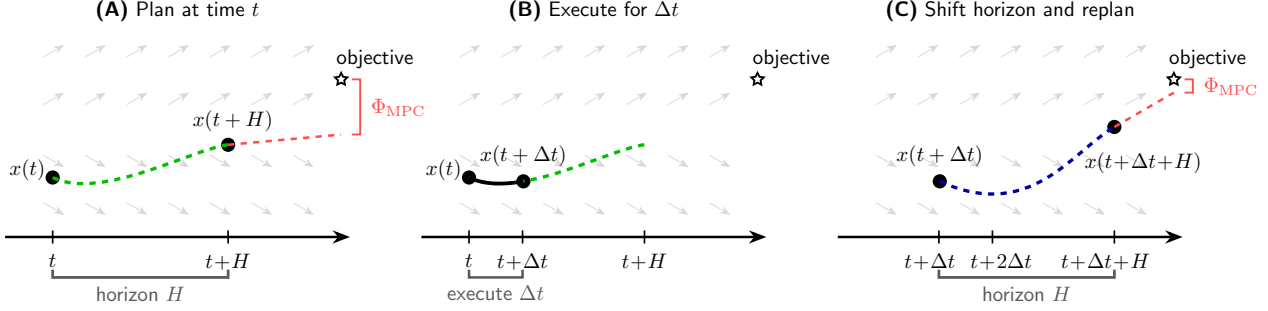


Figure 1. MPC-Flow strategy for guiding flow-based generative models toward a target objective. Starting from the current state, MPC-Flow plans a sequence of velocity adjustments that steer the flow toward the objective while keeping the intervention small (A). Only the initial part of the plan is applied (B), after which we re-plan from the new state and repeat the process (C).

In practice, the continuous-time optimal control problem in (5) is discretised. Both approaches proposed by Liu et al. (2023b) and Wang et al. (2025a), namely FlowGrad and OC-Flow, employ an explicit Euler discretisation with N uniform time steps of size $\Delta t = 1/N$, leading to the following discretised problem:

$$\begin{aligned} \min_{\mathbf{u}_0, \dots, \mathbf{u}_{N-1}} \quad & \frac{1}{N} \sum_{k=0}^{N-1} \|\mathbf{u}_k\|_2^2 + \lambda \Phi(\mathbf{x}_N) \\ \text{s.t.} \quad & \mathbf{x}_{t_{k+1}} = \mathbf{x}_{t_k} + \frac{1}{N} (v_\theta(\mathbf{x}_{t_k}, t_k) + \mathbf{u}_k), \\ & k = 1, \dots, N-1, \quad t_k = k/N. \end{aligned} \quad (6)$$

This discrete problem can, in principle, be solved using gradient descent, where the gradients for the controls $\mathbf{U} = [\mathbf{u}_0, \dots, \mathbf{u}_{N-1}]$ can be computed using automatic differentiation by backpropagating through the ODE solver, or using the adjoint method (Chen et al., 2018), which requires simulation of two ODEs (the forward and the adjoint) for every gradient computation, increasing the computational cost (i.e., runtime). Automatic differentiation incurs a high memory usage that scales linearly with the number of time steps, since computing the gradient with respect to $\mathbf{u}_k$ requires $N - k$ nested evaluations of the flow model v_θ , making it impractical for large-scale models. Check-pointing (Chen et al., 2016) is often employed to reduce memory usage, at the expense of increased computation time.

3. MPC-Flow

In this work, we propose MPC-Flow, a framework that leverages MPC (Garcia et al., 1989; Rawlings et al., 2020) for conditional generation with pre-trained flow models and addresses the computational bottlenecks identified in the previous paragraph.

MPC is an optimal control framework that has become standard in robotics and process control (Qin & Badgwell, 2003). At each time step, MPC computes a control by solving a finite-horizon optimal control problem initialised at the cur-

rent state. Specifically, at time t , MPC considers a planning horizon $[t, t + H]$ and computes an optimal control $\mathbf{u}(\tau)$ over this interval. Rather than executing the entire trajectory, only the first part, corresponding to a short interval $\Delta t < H$, is applied to the system. The state is then updated, the horizon is shifted forward, and the optimisation is repeated. In this work, the MPC problem solved at time t takes the form

$$\begin{aligned} \min_{\mathbf{u}} \quad & \int_t^{t+H} \|\mathbf{u}(\tau)\|^2 d\tau + \lambda \Phi_{\text{MPC}}(\mathbf{x}(t+H), t+H) \\ \text{s.t.} \quad & \frac{d\mathbf{x}(\tau)}{d\tau} = v_\theta(\mathbf{x}(\tau), \tau) + \mathbf{u}(\tau), \quad \mathbf{x}(t) = \hat{\mathbf{x}}_t, \end{aligned}$$

where $\hat{\mathbf{x}}_t$ is the current state at time t . MPC is defined by a choice of the horizon H , the number of discretisation steps K used to partition the horizon, and an intermediate cost function $\Phi_{\text{MPC}} : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}_{\geq 0}$. The overall approach is illustrated in Figure 1.

In particular, we consider two distinct regimes: (i) receding-horizon control with $H = 1 - t$; and (ii) Δt -horizon control with $H = \Delta t$. These choices correspond to two extreme cases. For $H = 1 - t$, the control is optimised over the full remaining trajectory, whereas for $H = \Delta t$ the optimisation is restricted to the next intermediate time step. In the following paragraphs, we analyse these two design choices and summarise their key differences in Table 1.

3.1. Receding-Horizon (RHC)

We first analyse the time horizon $H = 1 - t$, namely receding-horizon control (MPC-RHC). MPC-RHC re-plans the entire remaining trajectory at every step,

$$\begin{aligned} \mathbf{u}^* \in \arg \min_{\mathbf{u}} \quad & \int_t^1 \|\mathbf{u}(\tau)\|^2 d\tau + \lambda \Phi(\mathbf{x}(1)) \\ \text{s.t.} \quad & \frac{d\mathbf{x}(\tau)}{d\tau} = v_\theta(\mathbf{x}(\tau), \tau) + \mathbf{u}(\tau). \end{aligned} \quad (7)$$

The optimisation is carried out over $\tau \in [t, 1]$, with the state initialised as $\mathbf{x}(t) = \mathbf{x}_t$. The control is optimised over the remaining time horizon $[t, 1]$, but is only applied over a

Algorithm 1 Receding-Horizon Control (RHC)

input Pretrained flow model $v_\theta(\mathbf{x}, t)$, terminal cost $\Phi(\mathbf{x})$,
 initial state $\mathbf{x}_0$, number of coarse steps K , step size Δt
 1: $t' \leftarrow 0, \quad \mathbf{x} \leftarrow \mathbf{x}_0$
 2: **while** $t' < 1$ **do**
 3: Set $t_k \leftarrow t' + k(1 - t')/K$ # discretise $[t', 1]$
 4: Initialise control sequence $\mathbf{U} = [\mathbf{u}_0, \dots, \mathbf{u}_{K-1}]$
 5: **Solve the finite-horizon optimal control:**

$$\mathbf{U}^* = \arg \min_{\mathbf{U}} \left[\sum_{k=0}^{K-1} \frac{1-t'}{K} \|\mathbf{u}_k\|^2 + \lambda \Phi(\mathbf{x}_{t_K}) \right]$$

 subject to $\mathbf{x}_{t_{k+1}} = \mathbf{x}_{t_k} + \frac{1-t'}{K} (v_\theta(\mathbf{x}_{t_k}, t_k) + \mathbf{u}_k)$
 6: Apply only $\mathbf{u}_0^*$ on $[t', t' + \Delta t]$
 7: $\mathbf{x} \leftarrow \mathbf{x} + \Delta t (v_\theta(\mathbf{x}, t') + \mathbf{u}_0^*)$
 8: $t' \leftarrow t' + \Delta t$
 9: **end while**

Algorithm 2 Δt -Horizon Control

input Pretrained flow model $v_\theta(\mathbf{x}, t)$, MPC cost $\Phi_{\text{MPC}}(\mathbf{x})$,
 initial state $\mathbf{x}_0$, step size Δt
 1: $t' \leftarrow 0, \quad \mathbf{x} \leftarrow \mathbf{x}_0$
 2: $\Phi_{\text{MPC}}(\mathbf{x}, t) := \Phi(\mathbf{x} + (1 - t) v_\theta(\mathbf{x}, t))$
 3: **while** $t' < 1$ **do**
 4: Initialise control $\mathbf{u}$
 5: **Solve the one-step optimal control:**

$$\mathbf{u}^* = \arg \min_{\mathbf{u}} [\|\mathbf{u}\|^2 + \lambda \Phi_{\text{MPC}}(\mathbf{x}_{t'+\Delta t}, t' + \Delta t)]$$

 with $\mathbf{x}_{t'+\Delta t} = \mathbf{x}_{t'} + \Delta t (v_\theta(\mathbf{x}_{t'}, t') + \mathbf{u})$
 6: Apply $\mathbf{u}^*$ on $[t', t' + \Delta t]$
 7: $\mathbf{x} \leftarrow \mathbf{x} + \Delta t (v_\theta(\mathbf{x}, t') + \mathbf{u}^*)$
 8: $t' \leftarrow t' + \Delta t$
 9: **end while**

Figure 2. RHC optimises over the full horizon $[t, 1]$, discretised into K steps; Δt -Horizon uses a 1-step lookahead horizon $[t, t + \Delta t]$. For MPC-RHC, the original terminal loss Φ is used, whereas MPC- Δt employs the projected loss Φ_{MPC} .

short time horizon $[t, t + \Delta t]$ and the optimisation problem is re-solved from the updated state. As we solve the control problem up to the terminal time $t = 1$, the intermediate loss function is simply chosen to be $\Phi_{\text{MPC}}(\mathbf{x}, 1) := \Phi(\mathbf{x})$.

To analyse the theoretical properties of this scheme, we make use of the value function $V(t, \mathbf{x})$, defined as the minimum cost-to-go from state $\mathbf{x}$ at time t ,

$$V(t, \mathbf{x}) := \inf_{\mathbf{u}_{[t, 1]}} \left\{ \int_t^1 \|\mathbf{u}(\tau)\|^2 d\tau + \lambda \Phi(\mathbf{x}(1)) \right\},$$

where the infimum is taken with respect to the restricted interval $[t, 1]$. We can now formally state that the MPC-RHC recovers the global optimal control from (5).

Theorem 3.1 (Optimality of Receding-Horizon Control). *Let $\mathbf{u}^*$ be the global optimal control for the problem (5). If at every time t , the sub-problem (7) with $H = 1 - t$ and $\Phi_{\text{MPC}}(\cdot, 1) = \Phi$ is solved to optimality, then the sequence of applied controls coincides with $\mathbf{u}^*$.*

The proof follows directly from Bellman’s principle of optimality, see e.g. Liberzon (2011), and is provided in Appendix B.2.1 for reference. We validated this finding empirically with a 2D toy example, see Appendix C.

While Theorem 3.1 guarantees optimality, solving the MPC sub-problem over $[t, 1]$ remains, in principle, as expensive as solving the original problem (5). However, because we re-plan after every time step, MPC-RHC gains tractability through a coarse discretisation of the remaining horizon. At time t , we discretise $[t, 1]$ into K steps with grid points $t_k = t + k(1 - t)/K$ and approximate the controlled dynamics

Table 1. Comparison of MPC-Flow design choices.

Method	Horizon H	Discretisation	Backprop. through v_θ
Receding-Horizon Control (RHC) - Algorithm 1,3			
$K > 1$	$1 - t$	K steps	Yes ($\times K$)
$K = 1$	$1 - t$	1 step	No
Δt-Horizon Control - Algorithm 2			
MPC- Δt	Δt	1 step	Yes ($\times 1$)

using an explicit Euler scheme,

$$\mathbf{x}_{t_{k+1}} = \mathbf{x}_{t_k} + \frac{1-t}{K} [v_\theta(\mathbf{x}_{t_k}, t_k) + \mathbf{u}(t_k)].$$

This recursion is applied for $k = 0, \dots, K-1$, starting from $\mathbf{x}_{t_0} = \mathbf{x}_t$. As t increases, the remaining horizon shortens and the effective step size decreases, naturally yielding finer temporal resolution near the terminal time while allowing coarser discretisation earlier in the trajectory. The method is summarised in Algorithm 1; however, for $K > 1$, this variant may be unsuitable for real-time deployment, as solving the resulting sub-problem still requires backpropagation through $K-1$ nested evaluations of the flow model v_θ .

Single-Step RHC ($K=1$). A special case is to set $K=1$, i.e., solving the ODE on the remaining interval $[t, 1]$ using a single time step. For this choice the problem reduces to

$$\begin{aligned}
 \min_{\mathbf{u} \in \mathbb{R}^d} \{ \mathcal{J}_{K=1}(\mathbf{u}) &= (1-t') \|\mathbf{u}\|^2 + \lambda \Phi(\mathbf{x}') \}, \\
 \text{with } \mathbf{x}' &= \mathbf{x} + (1-t')(v_\theta(\mathbf{x}, t') + \mathbf{u}),
 \end{aligned}$$

This approximation effectively linearises the flow path. Crucially, the gradient $\nabla_{\mathbf{u}} \mathcal{J}_{K=1}(\mathbf{u})$ does not require backpropagating through the flow model v_θ . This dramatically reduces GPU memory requirements and computational cost,

making the method particularly suitable for scaling to large state-of-the-art architectures. This special case of Algorithm 1 is given in the Appendix, see Algorithm 3.

3.2. Δt -Horizon Control

Alternatively, we consider the greedy limit where the planning horizon is minimal, i.e., $H = \Delta t$. In this regime, we only optimise the control for the next immediate time step,

$$\begin{aligned} \mathbf{u}^* \in \arg \min_{\mathbf{u}} & \int_t^{t+\Delta t} \|\mathbf{u}(\tau)\|^2 d\tau \\ & + \lambda \Phi_{\text{MPC}}(\mathbf{x}(t + \Delta t), t + \Delta t) \quad (8) \\ \text{s.t. } & \frac{d\mathbf{x}(\tau)}{d\tau} = v_{\theta}(\mathbf{x}(\tau), \tau) + \mathbf{u}(\tau). \end{aligned}$$

The optimisation is performed over $\tau \in [t, t + \Delta t]$, with the state initialised at $\mathbf{x}(t) = \mathbf{x}_t$. This version of the MPC-Flow framework is described in Algorithm 2. Unlike for MPC-RHC, the choice of intermediate cost Φ_{MPC} is non-trivial here, as we are only solving the controlled ODE up to $t + \Delta t$. Simply using $\Phi(\mathbf{x})$ would result in a control that would fail to look ahead. We show that the optimal choice for the intermediate loss is the value function itself.

Theorem 3.2 (Δt -Horizon optimality). *The Δt -horizon control defined in (8) yields the globally optimal control policy if the terminal cost is chosen as the cost-to-go, i.e., $\Phi_{\text{MPC}}(\mathbf{x}, t + \Delta t) = V(t + \Delta t, \mathbf{x})$.*

The proof is again a consequence of the principle of optimality and provided in Appendix B.2.2. Since the true value function V is intractable, practical implementations of Δt -horizon control must rely on approximations. As a general-purpose heuristic, we can estimate the value function using a single Euler step approximation

$$V(t, \mathbf{x}) \approx \Phi(\mathbf{x} + (1 - t)v_{\theta}(\mathbf{x}, t)). \quad (9)$$

This approximates the remaining trajectory as a straight line without control. For an affine linear probability path, the single-step Euler prediction coincides with the Tweedie estimator used in diffusion models, see Proposition 1 of Kim et al. (2025). In diffusion-based settings, approximations of the value function such as (9) are commonly used. For instance, Li et al. (2024) and Uehara et al. (2025) employ such approximations within sequential Monte Carlo sampling schemes (Doucet et al., 2001). An alternative approach is to learn the value function explicitly, for example via soft Q-learning (Haarnoja et al., 2017). However, this requires an additional training phase and is therefore unsuitable for zero-shot applications.

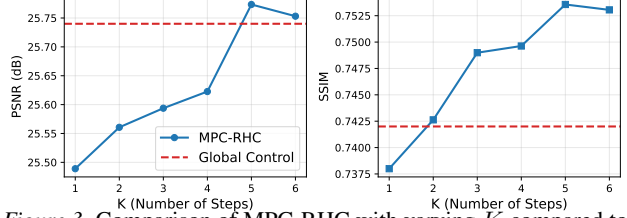


Figure 3. Comparison of MPC-RHC with varying K compared to the global optimal control solution. All approaches use the same initial value $\mathbf{x}_0$ and $\lambda = 2500$.

4. Experiments

In this section we perform experiments exploring the different design choices of MPC-Flow.¹ First, we tackle typical image restoration tasks, both with linear and non-linear degradation operators, on CelebA (Yang et al., 2015). Then, we show that we can scale MPC-RHC with $K = 1$ to the recently released FLUX.2 (Labs, 2025) model and consider style transfer and image colouration.

4.1. Computed Tomography

We train a flow-based model on the OrganCMNIST subset of MedMNIST (Yang et al., 2021; 2023), using the flow matching library (Lipman et al., 2024). The flow model is implemented as a UNet with approx. 8 million parameter, which allows us to directly implement the global optimal control problem (6) and compare the solution with MPC-RHC. In Figure 3 we compare the PSNR and SSIM of the final reconstruction for different choices of the discretisation K . For an increasing number of steps K we even get a higher PSNR and SSIM than the global control, which can be explained by the fact that the global solution is obtained via a non-convex optimisation problem and may converge to a suboptimal local minimum. We computed the global control using the same data-consistency strength λ and used the Adam optimiser (Kingma, 2014) until convergence. Further details and ablations are provided in Appendix D.

4.2. Image Restoration

We evaluate MPC-Flow on standard image restoration tasks using the CelebA dataset. All experiments use a pre-trained flow-matching model from Martin et al. (2025), trained with Mini-batch OT Flow (Tong et al., 2024) and a standard Gaussian latent prior. The flow model is implemented as a time-dependent U-Net (Ho et al., 2020). We primarily compare against optimal-control-based methods, specifically the global control approaches FlowGrad (Liu et al., 2023b) and OC-Flow (Wang et al., 2025a). We also include recent flow-based solvers for inverse problems: D-Flow (Ben-Hamu et al., 2024), OT-ODE (Pokle et al., 2024),

¹The code is publicly available at <https://github.com/alexdenker/MPCFlow>

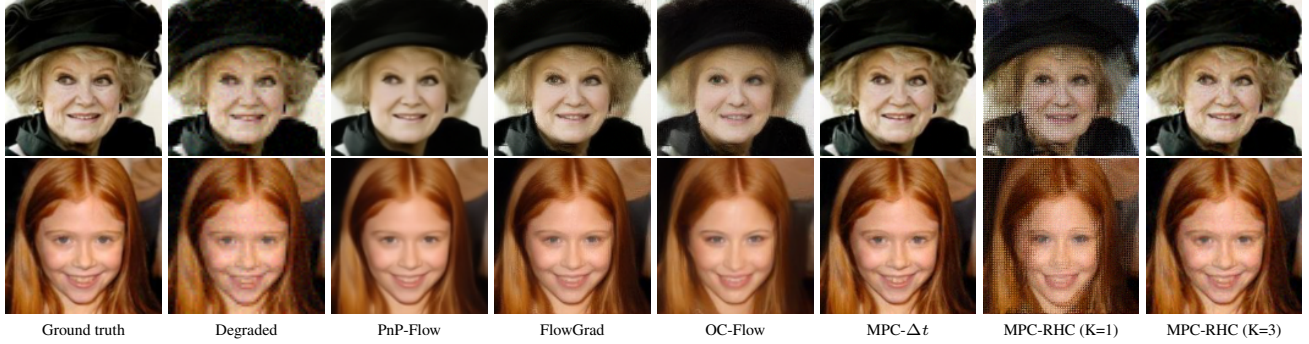


Figure 4. Qualitative results on the CelebA dataset for the image super-resolution task with noise level $\sigma = 0.05$ and $\times 2$ upscaling.

Method	Denoising $\sigma = 0.2$		Deblurring $\sigma = 0.05, \sigma_b = 1.0$		Super-res. $\sigma = 0.05, \times 2$		Rand. inpaint. $\sigma = 0.01, 70\%$		Box inpaint. $\sigma = 0.05, 40 \times 40$		Nonl. Debl. $\sigma = 0.05$		Time/img [s]
	PSNR $\uparrow$ / SSIM $\uparrow$		PSNR / SSIM		PSNR / SSIM		PSNR / SSIM		PSNR / SSIM		PSNR / SSIM		
Degraded	20.00	0.348	27.67	0.740	10.17	-0.182	11.82	-0.197	22.12	0.742	21.58	0.516	-
OT-ODE	30.50	0.867	32.63	0.915	31.05	0.902	28.36	0.865	28.84	0.914	21.95	0.626	5.19
D-Flow	26.42	0.651	31.07	0.877	30.75	0.866	33.07	0.938	29.70	0.893	26.41	<u>0.682</u>	26.31
Flow-Priors	29.26	0.766	31.40	0.856	28.35	0.717	32.33	0.945	29.40	0.858	21.84	0.490	24.30
PnP-Flow	32.45	0.911	34.51	0.940	<u>31.49</u>	<u>0.907</u>	33.54	0.953	<u>30.59</u>	0.943	22.19	0.643	5.97
FlowChef	21.57	0.403	26.02	0.596	27.68	0.729	28.46	0.848	26.41	0.776	17.59	0.279	1.50
FlowDPS	31.45	0.900	31.84	0.906	26.01	0.786	26.42	0.798	28.73	0.884	23.79	0.707	3.76
OC-Flow	19.39	0.559	27.81	0.828	24.95	0.746	16.77	0.456	26.97	0.856	20.17	0.528	234.76
FlowGrad	26.07	0.777	30.51	0.883	29.72	0.871	29.16	0.881	25.06	0.901	19.98	0.521	315.66
MPC- Δt	31.55	<u>0.877</u>	<u>33.65</u>	<u>0.928</u>	31.89	0.911	32.76	0.942	30.68	<u>0.931</u>	<u>25.44</u>	0.718	89.25
MPC-RHC (K=1)	27.30	0.708	32.47	0.900	18.02	0.421	18.31	0.455	26.13	<u>0.877</u>	17.63	0.405	2.05
MPC-RHC (K=3)	29.13	0.823	33.35	0.925	31.39	0.896	<u>33.25</u>	<u>0.951</u>	30.03	0.929	21.71	0.596	175.20

Table 2. Comparisons of state-of-the-art methods for several inverse problems on CelebA. Results are averaged across 100 test images. Reconstruction time is measured for the **Denoising** task. We compare general flow-based inverse problem solvers (OT-ODE, D-Flow, Flow-Priors PnP-Flow, FlowChef and FlowDPS), optimal control-based methods (FlowGrad, OC-Flow) and our MPC-Flow variants.

Flow-Priors (Zhang et al., 2024), PnP-Flow (Martin et al., 2025), FlowChef (Patel et al., 2025) and FlowDPS (Kim et al., 2025). Results are in Table E1 for linear image restoration tasks, including denoising, deblurring, super-resolution, random inpainting and box inpainting. Further, we provide results for non-linear deblurring. Qualitative results are in Figure 4. Additional results are in Appendix E.

Across most linear restoration tasks, PnP-Flow achieves the highest PSNR and SSIM, with MPC- Δt consistently ranking second. However, for super-resolution MPC- Δt achieves the best performance on both PSNR and SSIM. For non-linear deblurring, PnP-Flow degrades substantially. In this setting, D-Flow yields the highest PSNR, while MPC- Δt achieves the second-highest PSNR and the best SSIM. Across all tasks, MPC-Flow and its variants consistently outperform the global optimal control baselines FlowGrad and OC-Flow, while also achieving lower runtime. For completeness, FlowChef and FlowDPS were included in our evaluation, although both were originally designed for latent diffusion reconstruction tasks and lack explicit regularization strategies, resulting in suboptimal performance. In particular, FlowChef was primarily developed for image editing applications and is not well suited to high-noise im-

age restoration or image inpainting tasks, particularly in settings involving a non-trivial null space, as highlighted in (Erbach et al., 2026). The MPC-RHC variant with $K = 1$ performs poorly for some forward operators; this behaviour is discussed in Appendix E. This configuration corresponds to a direct implementation of Algorithm 3 without additional numerical tricks or heuristics. For each method, we conducted a hyperparameter search on a subset of the CelebA validation dataset and reported the used hyperparams in Appendix E.

4.3. Large Vision Models

Many recent large vision models (LVMs), including Stable Diffusion 3.0 (SD.3.0) (Esser et al., 2024) and FLUX.2 (Labs, 2025), are trained using flow-matching objectives and support both text and image conditioning. However, these models provide no guarantees for enforcing task-specific constraints. We apply our approach to two conditional generation tasks with FLUX.2: style transfer and image colouration. Due to the model’s scale (32B parameters), we use a 4-bit quantised version to reduce memory usage. Although quantisation is generally suitable for the forward pass, it significantly reduces the accuracy of backpropagation. Our

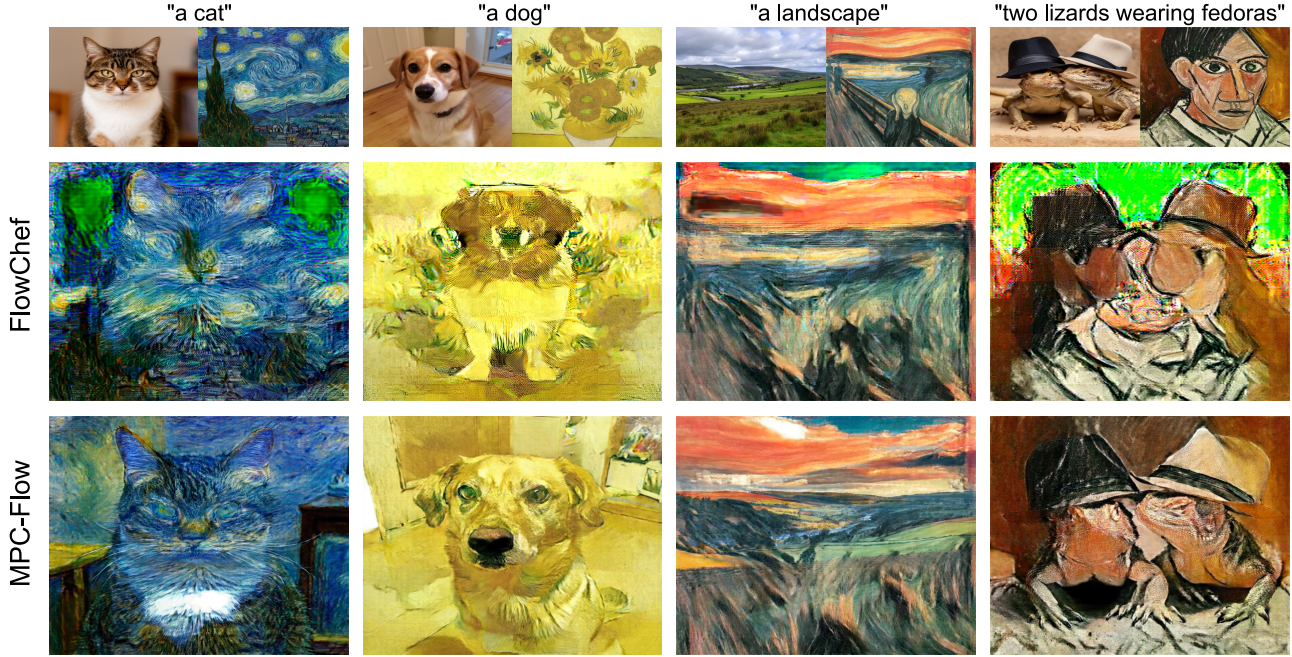


Figure 5. Example images generated by FLUX.2 with training-free style transfer guidance by FlowChef (middle row) and MPC (bottom row). The top row shows the image generated without conditioning as well as the prompt used and the reference style image.



Figure 6. Example re-colouration images generated with FLUX.2. Prompt used: “colourize this luminance image”.

single-step approach, MPC-RHC ($K = 1$), removes the need for backpropagation through the network, making it particularly suitable for LVM deployments. We compare our method to FlowChef (Patel et al., 2024), which can be viewed as a reparametrisation of our method without control regularisation. All experiments are run on an NVIDIA RTX 3090 GPU with only 24 GB of memory.

Style transfer We implemented our approach for style transfer, with the terminal loss

$$\Phi(\mathbf{z}) = \|\text{Style}(\text{Dec}(\mathbf{z})) - \text{Style}(\mathbf{x}_{\text{ref}})\|_2^2.$$

where the Style operator is defined via Gram matrices of CLIP ViT-B/16 (Radford et al., 2021) image features and $\mathbf{x}_{\text{ref}}$ is the reference style image. Figure 5 shows example images demonstrating qualitatively that on this task our approach delivered more realistic images. Figure 7 shows

different images generated with different hyper-parameters for the same prompt and style image. We fix a budget of 20 conditioning updates per time step. For our method, we vary the balance between terminal loss and control regularisation λ ; for FlowChef, we vary the learning rate on the terminal loss (see Appendix F). As the conditioning strength increases, FlowChef’s outputs move closer to the style image, but do so by substantially deviating from the original generation trajectory, which introduces pronounced artefacts. In contrast, our regularised control preserves the generation path more faithfully, allowing the image to adopt stylistic characteristics without severe degradation.

Figure 8 shows a quantitative summary across five prompts and nine style images. As hyperparameters vary, our method exhibits a more favourable trade-off between stylistic similarity and content preservation than FlowChef, consistently

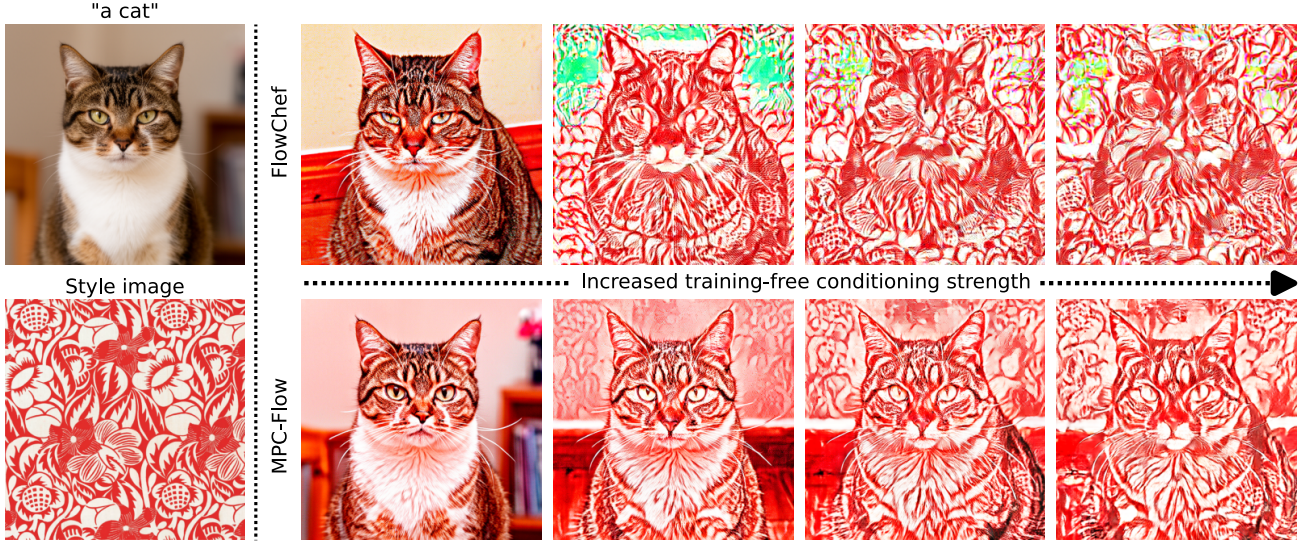


Figure 7. Column 1: (Top) An image generated by Flux.2. (Bottom) A reference style image. Remaining columns: Images generated using training-free style transfer with Flux.2 with FlowChef (top) and our method MPC-RHC ($K=1$) (bottom). From left to right, the methods use increased conditioning on the style reward; the path regularisation introduced by our method enables greater adherence to the original generation path (and hence closer alignment to the original image). For FlowChef, the learning rate of each style optimisation step was varied; for our method, we varied the amount of path regularisation (see Appendix F).

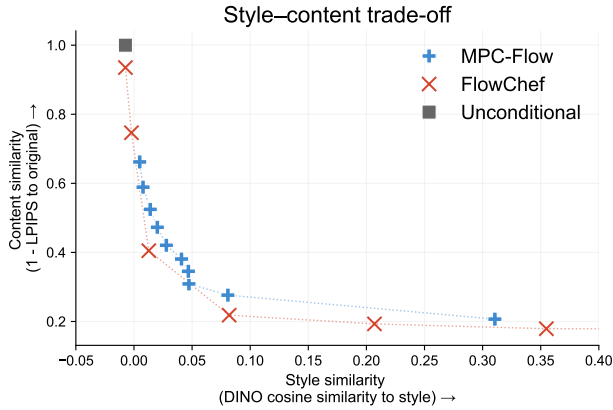


Figure 8. Style-content trade-off for MPC-Flow and FlowChef for style transfer with FLUX.2. Metrics recorded over 45 images (5 prompts $\times$ 9 style images).

lying on the superior side of the resulting Pareto frontier.

Image colouration We also considered the inverse problem of image colouration. Initially, FLUX.2 was prompted to colourise a provided luminance image, but struggled to remain faithful to the degradation process. We addressed this by introducing training-free guidance with MPC-Flow, using the terminal loss

$$\Phi(z) = \frac{1}{HW} \|\mathbf{x}_r - \ell(\mathbf{x})\|_2^2$$

where $\mathbf{x} = \text{Dec}(\mathbf{z}) \in [0, 1]^{3 \times H \times W}$ and luminance is defined as $\ell(\mathbf{x}) = 0.299 \mathbf{x}_R + 0.587 \mathbf{x}_G + 0.114 \mathbf{x}_B$.

Figure 6 shows example results for image colourisation.

Table 3. Results for image colouration with FLUX.2. Hyperparameters for FlowChef and MPC-Flow chosen to maximise PSNR.

Method	PSNR $\uparrow$	SSIM $\uparrow$
Unconditional	18.0	0.739
FlowChef	<u>21.3</u>	<u>0.844</u>
MPC-RHC ($K=1$)	23.8	0.891

Table 4. Computational cost for image colouration with FLUX.2. Generation used 28 flow steps and 20 conditioning updates/flow step.

Method	Peak VRAM (GB)	Time (s)
Unconditional	17.7	53
FlowChef	18.4	92
MPC-RHC ($K=1$)	18.4	101

Qualitatively, this shows that we can steer the flow towards an image that is consistent with the reference observation while remaining close to FLUX.2’s learned manifold, resulting in a higher-quality recoloured image. Image similarity metrics averaged over ten images and three random seeds are reported in Table 3. We additionally report the computational cost for this task with FLUX.2 in Table 4, demonstrating the modest overhead of using MPC-Flow (MPC-RHC ($K=1$)) relative to the unconditional baseline.

Super-resolution Lastly, we also implemented MPC-RHC ($K=1$) for the task of super-resolving a low-resolution image (analogously to the previous image colouration exam-

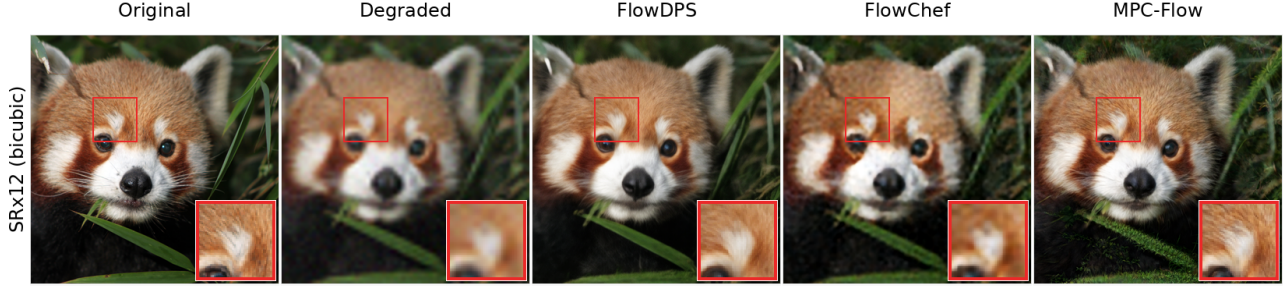


Figure 9. Qualitative comparison using SD3.0 on a bicubic $12\times$ SR example.

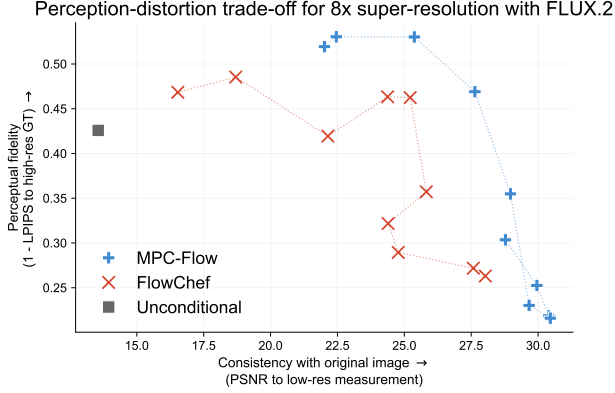


Figure 10. Perception-distortion trade-off for 8x super-resolution task with FLUX.2 (metrics averaged over 5 random seeds for a single image recovery task). Corresponding images are shown in Figure F1.

ple). The terminal loss used was a projected measurement-consistency loss in image space,

$$\Phi(\mathbf{z}) = \|\mathbf{A}^\dagger \mathbf{A} \text{Dec}(\mathbf{z}) - \mathbf{A}^\dagger \mathbf{y}\|_2^2, \quad \mathbf{y} = \mathbf{A} \mathbf{x}_{\text{GT}}.$$

where $\mathbf{A}$ is the $8\times$ bicubic downsampling operator, $\mathbf{A}^\dagger$ is bicubic upsampling back to the high-resolution grid, and $\mathbf{y}$ is the low-resolution measurement. Figure F1 shows an improved perception-distortion trade-off with MPC-Flow (MPC-RHC ($K = 1$)) relative to FlowChef and an unconditional baseline.

Additionally, to demonstrate the flexibility of our framework across latent flow models, we follow the noisy inverse-problem evaluation setup of FlowDPS (Kim et al., 2025) and we qualitatively evaluate MPC-Flow ($K = 1$) with SD3.0 (Esser et al., 2024) as the latent flow model. We consider three tasks: $12\times$ super-resolution (SR) from average pooling, $12\times$ super-resolution from bicubic interpolation, and Gaussian deblurring. A comparison for the bicubic $12\times$ SR setting is shown in Figure 9. Further details and visual results are provided in Appendix G, with full comparisons in Figure F3.

5. Conclusion & Further Work

We introduce MPC-Flow, a model predictive control framework for conditional generation with flow-based generative models. By decomposing the global trajectory optimisation into a sequence of control subproblems, MPC enables practical and memory-efficient guidance at inference time. We provide formal guarantees connecting MPC-Flow to the underlying optimal control problem. Empirically, MPC-Flow achieves strong performance across linear and non-linear image restoration tasks, while scaling to massive architectures such as FLUX.2 (32B) on consumer hardware. The single-step RHC variant in particular offers a highly efficient and scalable approach to inference-time control.

As a gradient-based method MPC-Flow requires access to a differentiable objective function. Further, the fast MPC-RHC with $K = 1$, does not always achieve good results on some linear inverse problems. MPC-Flow depends on the initial value $\mathbf{x}_0$ of the control problem, as discussed in Section H. In all our experiments we simply used a random initialisation for $\mathbf{x}_0$. However, more sophisticated initialisations are possible. For example, Ben-Hamu et al. (2024) initialise $\mathbf{x}_0$ by first integrating the dynamics backwards in time, starting at $t = 1$ with a naive reconstruction, e.g., computed using the adjoint of the forward operator.

We model the control as a simple additive vector to the flow dynamics. More expressive parametrisations are possible: for example, introducing non-linear control via $\dot{\mathbf{x}} = \mathbf{v}_\theta(\mathbf{x} + \mathbf{u}, t)$ (Pandey et al., 2025), or directly adapting the weights of the pre-trained model with $\dot{\mathbf{x}} = \mathbf{v}_{\theta+\mathbf{u}}(\mathbf{x}, t)$ using approaches such as low-rank adaptation (Hu et al., 2022) or control networks (Zhang et al., 2023). Further, the theoretical guarantees of Theorem 3.2 and Theorem 3.1 only apply to the continuous-time MPC formulation, while implemented methods introduce approximations: (i) time discretisation of the controlled dynamics via Euler integration; (ii) inexact numerical solution of each MPC subproblem and (iii) for MPC- Δt , approximation of the value function with a one-step surrogate. However, the discretisation error can be removed by adopting mean-flow models (Geng et al., 2026) instead of standard flow matching approaches.

Acknowledgments

AD and RB acknowledge support from the EPSRC (EP/V026259/1). AD additionally acknowledges support from DESY (Hamburg, Germany), a member of the Helmholtz Association HGF. GW acknowledges support from the EPSRC CDT in Smart Medical Imaging [EP/S022104/1] and by a GSK Studentship.

Impact Statement

This work improves inference-time control of flow-based generative models. As with other advances in generative modelling, increased controllability could be misused to generate harmful or misleading synthetic content, including deepfakes. The method does not introduce new generative capabilities beyond existing models and is intended for beneficial applications such as image restoration and scientific imaging.

References

- Albergo, M. S. and Vanden-Eijnden, E. Building normalizing flows with stochastic interpolants. In *The Eleventh International Conference on Learning Representations*, 2023.
- Barbano, R., Denker, A., Chung, H., Roh, T. H., Arridge, S., Maass, P., Jin, B., and Ye, J. C. Steerable conditional diffusion for out-of-distribution adaptation in medical image reconstruction. *IEEE Transactions on Medical Imaging*, 2025.
- Batzolis, G., Stanczuk, J., Schönlieb, C.-B., and Etmann, C. Conditional image generation with score-based diffusion models. *arXiv preprint arXiv:2111.13606*, 2021.
- Ben-Hamu, H., Puny, O., Gat, I., Karrer, B., Singer, U., and Lipman, Y. D-flow: Differentiating through flows for controlled generation. In *Forty-first International Conference on Machine Learning*, 2024.
- Berner, J., Richter, L., and Ullrich, K. An optimal control perspective on diffusion-based generative modeling. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856.
- Chen, R. T., Rubanova, Y., Bettencourt, J., and Duvenaud, D. K. Neural ordinary differential equations. *Advances in neural information processing systems*, 31, 2018.
- Chen, T., Xu, B., Zhang, C., and Guestrin, C. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*, 2016.
- Chung, H., Kim, J., Mccann, M. T., Klasky, M. L., and Ye, J. C. Diffusion posterior sampling for general noisy inverse problems. In *The Eleventh International Conference on Learning Representations*, 2023.
- Denker, A., Vargas, F., Padhy, S., Didi, K., Mathis, S., Barbano, R., Dutordoir, V., Mathieu, E., Komorowska, U. J., and Lio, P. Deft: Efficient fine-tuning of diffusion models by learning the generalised h -transform. *Advances in Neural Information Processing Systems*, 37:19636–19682, 2024.
- Dhariwal, P. and Nichol, A. Diffusion models beat gans on image synthesis. *Advances in neural information processing systems*, 34:8780–8794, 2021.
- Domingo-Enrich, C., Drozdal, M., Karrer, B., and Chen, R. T. Q. Adjoint matching: Fine-tuning flow and diffusion generative models with memoryless stochastic optimal control. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Doucet, A., De Freitas, N., Gordon, N. J., et al. *Sequential Monte Carlo methods in practice*, volume 1. Springer, 2001.
- Engl, H. W., Hanke, M., and Neubauer, A. *Regularization of Inverse Problems*. Mathematics and Its Applications. Springer, Dordrecht, 1996.
- Erbach, J., Narnhofer, D., Dombos, A. R., Schiele, B., Lenssen, J. E., and Schindler, K. Solving inverse problems with FLAIR. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026.
- Esser, P., Kulal, S., Blattmann, A., Entezari, R., Müller, J., Saini, H., Levi, Y., Lorenz, D., Sauer, A., Boesel, F., et al. Scaling rectified flow transformers for high-resolution image synthesis. In *Forty-first international conference on machine learning*, 2024.
- Fleming, W. H. and Rishel, R. W. *Deterministic and stochastic optimal control*, volume 1. Springer Science & Business Media, 2012.
- Garcia, C. E., Prett, D. M., and Morari, M. Model predictive control: Theory and practice—a survey. *Automatica*, 25 (3):335–348, 1989.
- Geng, Z., Deng, M., Bai, X., Kolter, J. Z., and He, K. Mean flows for one-step generative modeling. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026.
- Haarnoja, T., Tang, H., Abbeel, P., and Levine, S. Reinforcement learning with deep energy-based policies. In *International conference on machine learning*, pp. 1352–1361. PMLR, 2017.

- Ho, J., Jain, A., and Abbeel, P. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.
- Hu, E. J., yelong shen, Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- Kim, J., Kim, B. S., and Ye, J. C. FlowDPS: Flow-driven posterior sampling for inverse problems. *arXiv preprint arXiv:2503.08136*, 2025.
- Kingma, D. P. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- Labs, B. F. FLUX.2: Frontier Visual Intelligence. <https://bfl.ai/blog/flux-2>, 2025.
- Li, X., Zhao, Y., Wang, C., Scalia, G., Eraslan, G., Nair, S., Biancalani, T., Ji, S., Regev, A., Levine, S., et al. Derivative-free guidance in continuous and discrete diffusion models with soft value-based decoding. *arXiv preprint arXiv:2408.08252*, 2024.
- Li, Z., Alim, K., and Azizan, N. Hardflow: Hard-constrained sampling for flow-matching models via trajectory optimization. *arXiv preprint arXiv:2511.08425*, 2025.
- Liberzon, D. *Calculus of variations and optimal control theory: a concise introduction*. Princeton university press, 2011.
- Lipman, Y., Chen, R. T. Q., Ben-Hamu, H., Nickel, M., and Le, M. Flow matching for generative modeling. In *The Eleventh International Conference on Learning Representations*, 2023.
- Lipman, Y., Havasi, M., Holderrieth, P., Shaul, N., Le, M., Karrer, B., Chen, R. T., Lopez-Paz, D., Ben-Hamu, H., and Gat, I. Flow matching guide and code. *arXiv preprint arXiv:2412.06264*, 2024.
- Liu, Q. Rectified flow: A marginal preserving approach to optimal transport. *arXiv preprint arXiv:2209.14577*, 2022.
- Liu, X., Gong, C., and qiang liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *The Eleventh International Conference on Learning Representations*, 2023a.
- Liu, X., Wu, L., Zhang, S., Gong, C., Ping, W., and Liu, Q. Flowgrad: Controlling the output of generative odes with gradients. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 24335–24344, 2023b.
- Liu, Z., Xiao, T. Z., Domingo-Enrich, C., Liu, W., and Zhang, D. Value gradient guidance for flow matching alignment. *arXiv preprint arXiv:2512.05116*, 2025.
- Mardani, M., Song, J., Kautz, J., and Vahdat, A. A variational perspective on solving inverse problems with diffusion models. In *The Twelfth International Conference on Learning Representations*, 2024.
- Martin, S. T., Gagneux, A., Hagemann, P., and Steidl, G. Pnp-flow: Plug-and-play image restoration with flow matching. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Pandey, K., Sofian, F. M., Draxler, F., Karaletsos, T., and Mandt, S. Variational control for guidance in diffusion models. In *Forty-second International Conference on Machine Learning*, 2025.
- Patel, M., Wen, S., Metaxas, D. N., and Yang, Y. Steering rectified flow models in the vector field for controlled image generation. *arXiv preprint arXiv:2412.00100*, 2024.
- Patel, M., Wen, S., Metaxas, D. N., and Yang, Y. Flowchef: Steering of rectified flow models for controlled generations. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 15308–15318, 2025.
- Pokle, A., Muckley, M. J., Chen, R. T. Q., and Karrer, B. Training-free linear image inverses via flows. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856.
- Qin, S. J. and Badgwell, T. A. A survey of industrial model predictive control technology. *Control engineering practice*, 11(7):733–764, 2003.
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pp. 8748–8763. PmLR, 2021.
- Rawlings, J. B., Mayne, D. Q., Diehl, M., et al. *Model predictive control: theory, computation, and design*, volume 2. Nob Hill Publishing Madison, WI, 2020.
- Rout, L., Chen, Y., Ruiz, N., Kumar, A., Caramanis, C., Shakkottai, S., and Chu, W.-S. RB-modulation: Training-free stylization using reference-based modulation. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Song, B., Kwon, S. M., Zhang, Z., Hu, X., Qu, Q., and Shen, L. Solving inverse problems with latent diffusion models via hard data consistency. In *The Twelfth International Conference on Learning Representations*, 2024.

- Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., and Poole, B. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations*, 2021.
- Stuart, A. M. Inverse problems: a Bayesian perspective. *Acta numerica*, 19:451–559, 2010.
- Tong, A., FATRAS, K., Malkin, N., Huguët, G., Zhang, Y., Rector-Brooks, J., Wolf, G., and Bengio, Y. Improving and generalizing flow-based generative models with minibatch optimal transport. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856.
- Tran, P., Tran, A. T., Phung, Q., and Hoai, M. Explore image deblurring via encoded blur kernel space. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 11956–11965, 2021.
- Uehara, M., Zhao, Y., Wang, C., Li, X., Regev, A., Levine, S., and Biancalani, T. Inference-time alignment in diffusion models with reward-guided generation: Tutorial and review. *arXiv preprint arXiv:2501.09685*, 2025.
- Vargas, F., Grathwohl, W. S., and Doucet, A. Denoising diffusion samplers. In *The Eleventh International Conference on Learning Representations*, 2023.
- von Platen, P., Patil, S., Lozhkov, A., Cuenca, P., Lambert, N., Rasul, K., Davaadorj, M., Nair, D., Paul, S., Berman, W., Xu, Y., Liu, S., and Wolf, T. Diffusers: State-of-the-art diffusion models. <https://github.com/huggingface/diffusers>, 2022.
- Wang, H., Zhang, X., Li, T., Wan, Y., Chen, T., and Sun, J. Dmplug: A plug-in method for solving inverse problems with diffusion models. *Advances in Neural Information Processing Systems*, 37:117881–117916, 2024.
- Wang, L., Cheng, C., Liao, Y., Qu, Y., and Liu, G. Training free guided flow-matching with optimal control. In *The Thirteenth International Conference on Learning Representations*, 2025a.
- Wang, Y., Lu, J., Jaitly, N., Susskind, J., and Bautista, M. A. Simplefold: Folding proteins is simpler than you think. *arXiv preprint arXiv:2509.18480*, 2025b.
- Yang, J., Shi, R., and Ni, B. Medmnist classification decathlon: A lightweight automl benchmark for medical image analysis. In *IEEE 18th International Symposium on Biomedical Imaging (ISBI)*, pp. 191–195, 2021.
- Yang, J., Shi, R., Wei, D., Liu, Z., Zhao, L., Ke, B., Pfister, H., and Ni, B. MedMNIST v2-a large-scale lightweight benchmark for 2d and 3d biomedical image classification. *Scientific Data*, 10(1):41, 2023.
- Yang, S., Luo, P., Loy, C.-C., and Tang, X. From facial parts responses to face detection: A deep learning approach. In *Proceedings of the IEEE international conference on computer vision*, pp. 3676–3684, 2015.
- Zhang, B., Chu, W., Berner, J., Meng, C., Anandkumar, A., and Song, Y. Improving diffusion inverse problem solving with decoupled noise annealing. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 20895–20905, 2025.
- Zhang, L., Rao, A., and Agrawala, M. Adding conditional control to text-to-image diffusion models. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 3836–3847, 2023.
- Zhang, Q. and Chen, Y. Path integral sampler: A stochastic control approach for sampling. In *International Conference on Learning Representations*, 2022.
- Zhang, Y., Yu, P., Zhu, Y., Chang, Y., Gao, F., Wu, Y. N., and Leong, O. Flow priors for linear inverse problems via iterative corrupted trajectory matching. *Advances in Neural Information Processing Systems*, 37:57389–57417, 2024.

A. Related Work

Continuous Normalising Flows Continuous Normalising Flows (Chen et al., 2018) trained with Flow Matching (Lipman et al., 2023) have emerged as a powerful methods for generating high-dimensional data, for example for images (Esser et al., 2024) or biological data (Wang et al., 2025b). A key advantage of Flow Matching is its ability to learn Gaussian probability paths corresponding to Optimal Transport (OT) displacements. These paths yield straighter trajectories, enabling faster sampling (Albergo & Vanden-Eijnden, 2023; Liu et al., 2023a) compared to standard diffusion models (Song et al., 2021). However, as training these models remains computationally expensive, there is significant interest in leveraging pre-trained unconditional models as priors for downstream conditional tasks, such as inpainting, super-resolution, or deblurring in imaging inverse problems.

Existing approaches for conditional generation with continuous-time models generally fall into two categories. The first involves conditional training and fine-tuning, where a model is either trained from scratch on paired data (Batzolis et al., 2021) or the weights of the model are adapted (Domingo-Enrich et al., 2025; Zhang et al., 2023). These approaches rely on the availability of additional data and/or require a costly fine-tuning phase. The second category consists of training-free guidance methods, which modify the sampling process to satisfy conditions without altering the unconditional model (Kim et al., 2025; Pogle et al., 2024; Zhang et al., 2024). It is this category that is the subject of this work.

Guidance for Continuous-time Generative Models The problem of steering the generative processes has been heavily studied in the context of diffusion models (Song et al., 2021). This can be framed as Stochastic Optimal Control (SOC), which has been used in the context of inverse problems (Denker et al., 2024; Pandey et al., 2025), fine-tuning (Domingo-Enrich et al., 2025) and model personalisation (Rout et al., 2025). Further, SOC methods have been applied to sampling from unnormalised probability density functions (Zhang & Chen, 2022; Berner et al., 2024; Vargas et al., 2023). These methods rely on stochastic calculus and score-based dynamics, which fundamentally differ from the deterministic ODEs induced by Flow Matching. As an alternative, training-free guidance methods, such as DPS (Chung et al., 2023), ReSample (Song et al., 2024) or DAPS (Zhang et al., 2025), modify the sampling dynamics using approximations of conditional scores. Several of these ideas have been adapted to flow-based models, for example RED-diff (Mardani et al., 2024) was first proposed for diffusion models and later adapted to flow-based models (Erbach et al., 2026). Other approaches are specific to flow-based models; for example, FlowChef (Patel et al., 2024) exploits the deterministic structure of rectified flows to perform gradient-free steering by directly manipulating the vector field during sampling. These methods can be applied at inference time without changing the underlying pre-trained model. However, they typically only provide limited guarantees: the final sample may not satisfy the condition exactly, and the trade-off between consistency to observations and staying close to the original trajectory is only implicit.

Optimal Control in Flow Models Applying optimal control to flow-based models is an active area of research. Existing approaches differ mainly in how gradients are computed, how control cost is handled, and the associated trade-offs of memory and computation time. To mitigate the memory cost associated with large scale optimisation problems, several works utilise adjoint sensitivity methods (Chen et al., 2018). FlowGrad (Liu et al., 2023b) leverages a time-discrete version of the adjoint ODE to compute control gradients. To reduce the computational cost, they introduce a metric for ODE straightness, allowing for fewer backward time steps.

However, this approach still requires solving the full forward ODE at each optimisation step, and notably, the running cost of the control is excluded from the gradient computation. Similarly, OC-Flow (Wang et al., 2025a) employs a time-discrete adjoint for gradient computation but implements a control regularisation implicitly via weight decay.

Both FlowGrad and OC-Flow focus primarily on image editing tasks. D-Flow (Ben-Hamu et al., 2024) only optimises the initial value of the ODE, without introducing additional control terms. Unlike adjoint-based approaches, D-Flow directly backpropagates gradients through the discrete ODE solver. While this avoids adjoint approximation errors, it drastically increases the memory cost as the full trajectory and all model evaluations have to be kept in memory. Finally, VGG-Flow (Liu et al., 2025) estimates the value function gradient and fine-tunes the underlying flow-based model via a value gradient matching loss. From these above-mentioned models only D-Flow has been applied to inverse problems.

Concurrently, HardFlow (Li et al., 2025) introduces a training-free framework for hard-constrained sampling based on receding-horizon model predictive control, closely related to the approach proposed in this work.

B. Proofs

B.1. Existence of Optimal Control

Following the standard literature, see e.g. (Fleming & Rishel, 2012), we give an existence proof of the optimal control for our setting.

Proposition B.1 (Existence of optimal control). *Under the assumptions that*

- 1) v_θ is continuous in both arguments
- 2) v_θ is uniformly Lipschitz in t , i.e., $\|v_\theta(\mathbf{x}_1, t) - v_\theta(\mathbf{x}_2, t)\| \leq L\|\mathbf{x}_1 - \mathbf{x}_2\|$ for some $L > 0$,
- 3) v_θ has at most linear growth, i.e., $\|v_\theta(\mathbf{x}, t)\| \leq a(t) + b\|\mathbf{x}\|$ with $a \in L^1([0, 1])$ and $b > 0$,
- 4) $\Phi : \mathbb{R}^n \rightarrow \mathbb{R}$ is lower semicontinuous,

there exists an optimal control $\mathbf{u}^* \in L^2([0, 1])$ of the control problem

$$\begin{aligned} \min_{\mathbf{u}} \left\{ \mathcal{J}(\mathbf{u}) := \int_0^1 \|\mathbf{u}(t)\|^2 dt + \Phi(\mathbf{x}(1)) \right\} \\ \text{s.t. } \quad d\mathbf{x}(t)/dt = v_\theta(\mathbf{x}(t), t) + \mathbf{u}(t), \\ t \in [0, 1], \mathbf{x}(0) = \mathbf{x}_0. \end{aligned} \quad (10)$$

Proof. First, we observe that the infimum is finite, i.e., $\inf \mathcal{J}(\mathbf{u}) < \infty$, as $\mathbf{u} = 0$ is an admissible control and we have the growth condition on v_θ . Let $(\mathbf{u}_k)$ be a minimising sequence such that $\mathcal{J}(\mathbf{u}_k) \rightarrow \inf \mathcal{J}(\mathbf{u})$. As the control cost $\int_0^1 \|\mathbf{u}_k(t)\|^2 dt$ is bounded, we have a weakly convergent subsequence (which we still denote it as $\mathbf{u}_k$ for an easier notation), such that $\mathbf{u}_k \rightharpoonup \mathbf{u}^*$ in $L^2([0, 1])$. We have to show that the optimal candidate $\mathbf{u}^*$ is actually a minimiser, i.e., $\mathcal{J}(\mathbf{u}^*) = \inf \mathcal{J}(\mathbf{u})$.

For this, we first look at the convergence of the trajectories. Let $\mathbf{x}_k$ and $\mathbf{x}^*$ denote the trajectories corresponding to $\mathbf{u}_k$ and $\mathbf{u}^*$ respectively. We can express the trajectories as

$$\mathbf{x}_k(t) = \mathbf{x}_0 + \int_0^t v_\theta(\mathbf{x}_k(s), s) ds + \int_0^t \mathbf{u}_k(s) ds, \quad \mathbf{x}^*(t) = \mathbf{x}_0 + \int_0^t v_\theta(\mathbf{x}^*(s), s) ds + \int_0^t \mathbf{u}^*(s) ds. \quad (11)$$

Let us define the integrated control error as $E_k(t) := \int_0^t (\mathbf{u}_k(s) - \mathbf{u}^*(s)) ds$. Since $\mathbf{u}_k \rightharpoonup \mathbf{u}^*$ weakly in $L^2([0, 1])$, we obtain $E_k(t) \rightarrow 0$ for any fixed t . Furthermore, as the controls $(\mathbf{u}_k)$ are bounded, we have that $E_k(t)$ is bounded. By the Arzelà-Ascoli, there is then a subsequence of E_k which converges uniformly to 0, i.e., there is $\epsilon_k := \sup_{t \in [0, 1]} \|E_k\| \rightarrow 0$.

Further, we get for the trajectories

$$\|\mathbf{x}_k(t) - \mathbf{x}^*(t)\| \leq \int_0^t \|v_\theta(\mathbf{x}_k(s), s) - v_\theta(\mathbf{x}^*(s), s)\| ds + \|E_k(t)\| \quad (12)$$

$$\leq \int_0^1 L \|\mathbf{x}_k(s) - \mathbf{x}^*(s)\| + \epsilon_k \quad (13)$$

$$\leq \epsilon_k \exp(Lt) \quad (14)$$

where the last inequality is due to Grönwall. As $\epsilon_k \rightarrow 0$, we have that $\mathbf{x}_k(t) \rightarrow \mathbf{x}^*(t)$ uniformly on $[0, 1]$. In particular, $\mathbf{x}_k(1) \rightarrow \mathbf{x}^*(1)$.

Finally, we get

$$\mathcal{J}(\mathbf{u}^*) = \int_0^1 \|\mathbf{u}^*(t)\|^2 dt + \Phi(\mathbf{x}^*(1)) \leq \liminf_{k \rightarrow \infty} \left(\int_0^1 \|\mathbf{u}_k(t)\|^2 dt + \Phi(\mathbf{x}_k(1)) \right) = \inf \mathcal{J}(\mathbf{u}), \quad (15)$$

as both the squared L^2 norm and Φ are lower semicontinuous. Thus, $\mathbf{u}^*$ is an optimal control. $\square$

Importantly, we do not need that the control is in some compact set due to the control cost $\int_0^1 \|\mathbf{u}(t)\|^2 dt$.

B.2. Optimality of MPC

B.2.1. PROOF OF THEOREM 3.1

Proof. This result follows directly from Bellman’s principle of optimality, see e.g. Liberzon (2011). Let the optimal trajectory be $\mathbf{x}^*$. Suppose we are at time t in state $\mathbf{x}^*(t)$. The global cost can be split

$$\mathcal{J}(\mathbf{u}) = \int_0^t \|\mathbf{u}\|^2 d\tau + \int_t^1 \|\mathbf{u}\|^2 d\tau + \lambda\Phi(\mathbf{x}(1)).$$

Since the past cost in $[0, t]$ is fixed, minimising the global cost is equivalent to minimising the future cost, i.e. the second and third terms. The solution to the sub-problem (7) at time t is exactly the restriction of the global optimal control to the interval $[t, 1]$. Thus, re-optimising at each step yields the same control as (5). $\square$

B.2.2. PROOF OF THEOREM 3.2

Proof. The value function $V(t, \mathbf{x}(t))$ can be expressed as

$$\min_{\mathbf{u}} \left\{ \int_t^{t+\Delta t} \|\mathbf{u}(\tau)\|^2 d\tau + V(t+\Delta t, \mathbf{x}(t+\Delta t)) \right\}, \quad (16)$$

for a time step Δt . This optimisation problem is identical to (8) with $\Phi_{\text{MPC}}(\mathbf{x}(t+\Delta t)) = V(t+\Delta t, \mathbf{x}(t+\Delta t))$. This means that minimising (8) for this choice of terminal cost evaluates the value function at every time step and thus we obtain the optimal control. $\square$

C. Validating optimality guarantees with a toy example

In Theorem 3.1, we establish the optimality of the receding-horizon control (RHC) strategy under the assumption that each finite-horizon sub-problem is solved exactly. We complement this theoretical result with an empirical validation on a simple two-dimensional controlled flow-matching example, where the flow trajectories can be visualised directly.

We train a flow model using flow matching to generate samples lying on the boundary of a regular hexagon with side length 2. Training pairs are constructed by sampling $\mathbf{x}_0 \sim \mathcal{N}(0, I)$ and $\mathbf{x}_1$ uniformly from the hexagon boundary, and defining an affine interpolation path between $\mathbf{x}_0$ and $\mathbf{x}_1$. The model then learns a time-dependent velocity field $\mathbf{v}(\mathbf{x}_t, t)$ that matches the instantaneous velocity of this affine path at each time step (see equation (4)). Integrating $\mathbf{v}$ thus transports isotropic noise toward the hexagon boundary along approximately straight-line trajectories.

At inference time, we apply MPC to steer the generative trajectory toward a specific mode of the distribution. In particular, the terminal loss is chosen as

$$\Phi(\mathbf{x}) = \|\mathbf{x} - \mathbf{x}_{\text{corner}}\|_2^2,$$

where $\mathbf{x}_{\text{corner}}$ denotes the coordinates of the lower-right corner of the hexagon. While the learned flow alone produces samples uniformly along the hexagon’s boundary, the MPC controller biases the otherwise affine transport path toward the selected corner, providing a concrete and interpretable setting in which to assess the optimality guarantees of Theorem 3.1.

MPC-RHC We first present results for MPC-RHC in Figure C1. Recall that at time step t , MPC-RHC plans the remaining trajectory as an Euler discretisation of $\mathbf{x}_{[t,1]}$ with K steps. It may be observed that the MPC-RHC methods approximate the globally optimal control as K increases. Further, all methods achieve the desired target point while balancing control energy and the terminal loss. Figure C2a demonstrates empirically that the error to the globally optimal control decreases as K increases, while Figure C2b demonstrates that as K increases, the method finds a stable balance between control regularisation and satisfying the terminal objective.

MPC- Δt For MPC- Δt we use a single Euler step approximation for the value functions, see (9). For this 2D example, we can numerically solve the Hamilton–Jacobi–Bellman (HJB) equation using a semi-Lagrangian scheme that propagates the value function. This yields both the value function and the optimal control. In Figure C3 we compare this true value function against the one step approximation. At early times ($t \approx 0$) the discrepancy is large, but it decreases as time progresses. Due to this discrepancy at small times we do not recover the global optimal control. However, as time increases the MPC- Δt control follows the global control, see also (f) in Figure C3.

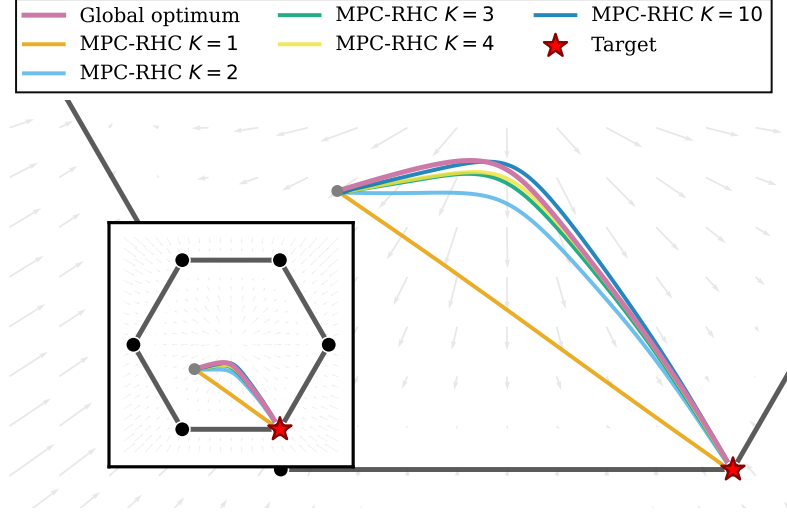


Figure C1. MPC-RHC for the 2D hexagon toy example. The flow model is trained to transport random points to the hexagon boundary. MPC-RHC’s terminal loss is $\Phi(\mathbf{x}) = \|\mathbf{x} - \mathbf{x}_{\text{corner}}\|^2$, where $\mathbf{x}_{\text{corner}}$ are the coordinates of the lower right corner of the hexagon. The “global optimum” path represents the exact solution to the optimal control problem (solved globally over the whole trajectory $\mathbf{x}_{[0,1]}$).

D. Computed Tomography

We make use of the OrganCMNIST subset of MedMNIST (Yang et al., 2021; 2023) to provide ablations on the hyperparameters of the different variations of MPC-Flow. The OrganCMNIST dataset contains 64×64 px adnominal CT images. In total there are 23 582 images, split into 12 975 training, 2392 validation and 8216 test images. We train an unconditional flow model on this dataset. Here, we make use of the flow matching library (Lipman et al., 2024). The flow model is implemented as a time-dependent UNet with approx. 8Mio parameters (Dhariwal & Nichol, 2021). For the forward operator we make use of the parallel-beam Radon transform with 18 angles and corrupt the simulated measurements with 1% relative additive Gaussian noise.

MPC- Δt We first provide ablations for MPC- Δt , see Algorithm 2. For every t' , we have to solve the optimisation problem

$$\begin{aligned} \mathbf{u}^* &= \arg \min_{\mathbf{u}} [\|\mathbf{u}\|^2 + \lambda \Phi_{\text{MPC}}(\mathbf{x}_{t'+\Delta t}, t' + \Delta t)] \\ &\quad \text{with } \mathbf{x}_{t'+\Delta t} = \mathbf{x}_{t'} + \Delta t(v_{\theta}(\mathbf{x}_{t'}, t') + \mathbf{u}), \end{aligned}$$

with $\lambda = \tilde{\lambda}/\Delta t$, where the intermediate loss is defined as

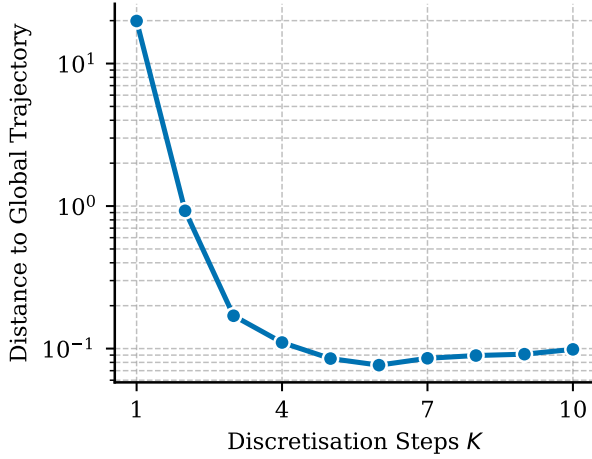
$$\Phi_{\text{MPC}}(\mathbf{x}_{t'+\Delta t}, t' + \Delta t) := \Phi(\mathbf{x}_{t'+\Delta t} + (1 - (t' + \Delta t))v_{\theta}(\mathbf{x}_{t'+\Delta t}, t' + \Delta t)).$$

We solve this optimisation problem using the Adam optimiser (Kingma, 2014). For these experiments, we use a fixed learning rate of 0.1 and re-initialise the optimiser with $\mathbf{u}_{\text{init}} = \mathbf{0}$ at every new time step. We vary both the data-consistency strength λ and the step size $\Delta t = 1/N$. In Figure D1a we study the effect of the data-consistency strength λ , while keeping $N = 40$ constant. Here, we observe that the optimal value, both for the PSNR and SSIM, is at $\lambda \approx 1 \times 10^4$ and then degrades for larger values. Further, for a fixed $\lambda = 1 \times 10^4$ we show the dependency on the discretisation N in Figure D1b. For a coarser discretisation, the flow model has less ability to explore fine-grained modes of the image distribution, and returns a smoother image; PSNR favours smoother images over incorrectly detailed images, so PSNR rises as N decreases in this case. We show exemplary reconstruction in Figure D2.

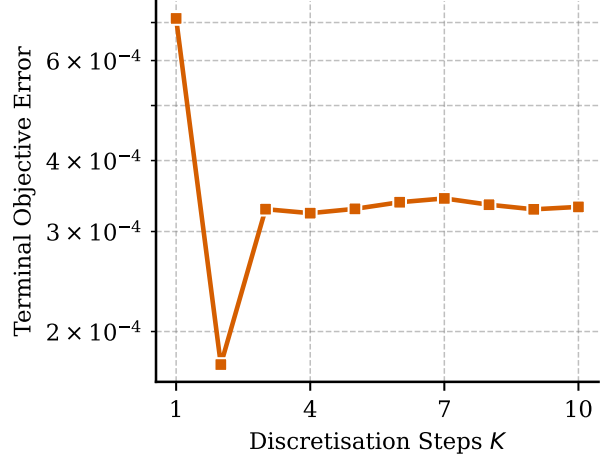
E. Image Restoration

This section complements the experimental study on linear imaging tasks (denoising, deblurring, super-resolution, random inpainting, and box inpainting) and a nonlinear imaging task (nonlinear deblurring).

We report qualitative results for the best-performing methods (or those most suitable for comparison with our proposed approach) across each image restoration task; see Figures E1 to E6 for results on the CelebA test set.



(a) Distance to the globally optimal trajectory for different discretisation steps K .



(b) Terminal error of MPC-RHC trajectories for different discretisation steps K .

Figure C2. Effect of discretisation steps K on MPC-RHC performance.

We also report the hyperparameters used in each experiment. Hyperparameters were selected on a subset of the CelebA validation split. Table E2 summarises the hyperparameters for FlowGrad, OC-Flow, FlowChef and FlowDPS, along with our proposed methods.

For the linear restoration tasks, the hyperparameters of OT-ODE, D-Flow, Flow-Priors, and PnP-Flow are taken from Martin et al. (2025). We use 100 time steps for all tasks. For the nonlinear task, hyperparameters were selected on a subset of the validation split.

Note that for the nonlinear deblurring task (see Table E1), we adopt the nonlinear forward operator used in Chung et al. (2023); Denker et al. (2024); Wang et al. (2024). The operator is parameterised by a trained neural network (Tran et al., 2021), inducing a highly nonlinear blur. Our implementation follows the publicly available codebase of Blur-Kernel-Space-Exploring².

Method	Denoising $\sigma = 0.2$			Deblurring $\sigma = 0.05, \sigma_b = 1.0$			Super-res. $\sigma = 0.05, \times 2$			Rand. inpaint. $\sigma = 0.01, 70\%$			Box inpaint. $\sigma = 0.05, 40 \times 40$			Nonl. Debl. $\sigma = 0.05$			Time/img [s]
	PSNR $\uparrow$ / SSIM $\uparrow$ / LPIPS $\downarrow$			PSNR / SSIM / LPIPS			PSNR / SSIM / LPIPS			PSNR / SSIM / LPIPS			PSNR / SSIM / LPIPS			PSNR / SSIM / LPIPS			
Degraded	20.00	0.348	0.373	27.67	0.740	0.125	10.17	0.182	0.827	11.82	0.197	1.034	22.12	0.742	0.213	21.58	0.516	0.370	–
OT-ODE	30.50	0.867	0.110	32.63	0.915	0.096	31.05	0.902	0.091	28.36	0.865	0.137	28.84	0.914	0.102	21.95	0.626	0.118	5.19
D-Flow	26.42	0.651	0.078	31.07	0.877	0.053	30.75	0.866	0.027	33.07	0.938	0.021	29.70	0.893	0.035	26.41	0.682	0.069	26.31
Flow-Priors	29.26	0.766	0.137	31.40	0.856	0.056	28.35	0.717	0.101	32.33	0.945	0.018	29.40	0.858	0.048	21.84	0.490	0.318	24.30
PnP-Flow	32.45	0.911	0.056	34.51	0.940	0.046	31.49	0.907	0.055	33.54	0.953	0.021	30.59	0.943	0.043	22.19	0.643	0.263	5.97
FlowChef	21.57	0.403	0.301	26.02	0.596	0.208	27.68	0.729	0.085	28.46	0.848	0.061	26.41	0.776	0.067	17.59	0.279	0.451	1.50
FlowDPS	31.45	0.900	0.041	31.84	0.906	0.043	26.01	0.786	0.118	26.42	0.798	0.108	28.73	0.884	0.063	23.79	0.707	0.157	3.76
OC-Flow	19.39	0.559	0.207	27.81	0.828	0.082	24.95	0.746	0.114	16.77	0.456	0.418	26.97	0.856	0.075	20.17	0.528	0.170	234.76
FlowGrad	26.07	0.777	0.116	30.51	0.883	0.067	29.72	0.871	0.050	29.16	0.881	0.039	25.06	0.901	0.068	19.98	0.521	0.170	315.66
MPC- Δt	31.55	0.877	0.029	33.65	0.928	0.017	31.89	0.911	0.018	32.76	0.942	0.015	30.68	0.931	0.021	25.44	0.718	0.067	89.25
MPC-RHC ($K=1$)	27.30	0.708	0.092	32.47	0.900	0.030	18.02	0.421	0.325	18.31	0.455	0.419	26.13	0.877	0.046	17.63	0.405	0.244	2.05
MPC-RHC ($K=3$)	29.13	0.823	0.066	33.35	0.925	0.023	31.39	0.896	0.029	33.25	0.951	0.013	30.03	0.929	0.026	21.71	0.596	0.133	175.20

Table E1. Comparisons of state-of-the-art methods for several inverse problems on CelebA. Results are averaged across 100 test images. Reconstruction time is measured for the **Denoising** task. We compare general flow-based inverse problem solvers (OT-ODE, D-Flow, Flow-Priors, PnP-Flow, FlowChef and FlowDPS), optimal control-based methods (FlowGrad, OC-Flow) and our MPC-Flow variants.

E.1. MPC-RHC $K = 1$ for Linear Inverse Problems

A limitation of the single-step approach emerges when applied to linear inverse problems where the forward operator $\mathbf{A}$ has a large null space. Typical examples include inpainting or certain instances of super-resolution, where some pixels are completely unobserved. Consider the case with a quadratic terminal loss $\Phi(\mathbf{x}) = \frac{1}{2} \|\mathbf{A}\mathbf{x} - \mathbf{y}\|_2^2$. The MPC-RHC $K = 1$

²<https://github.com/VinAIRResearch/blur-kernel-space-exploring>

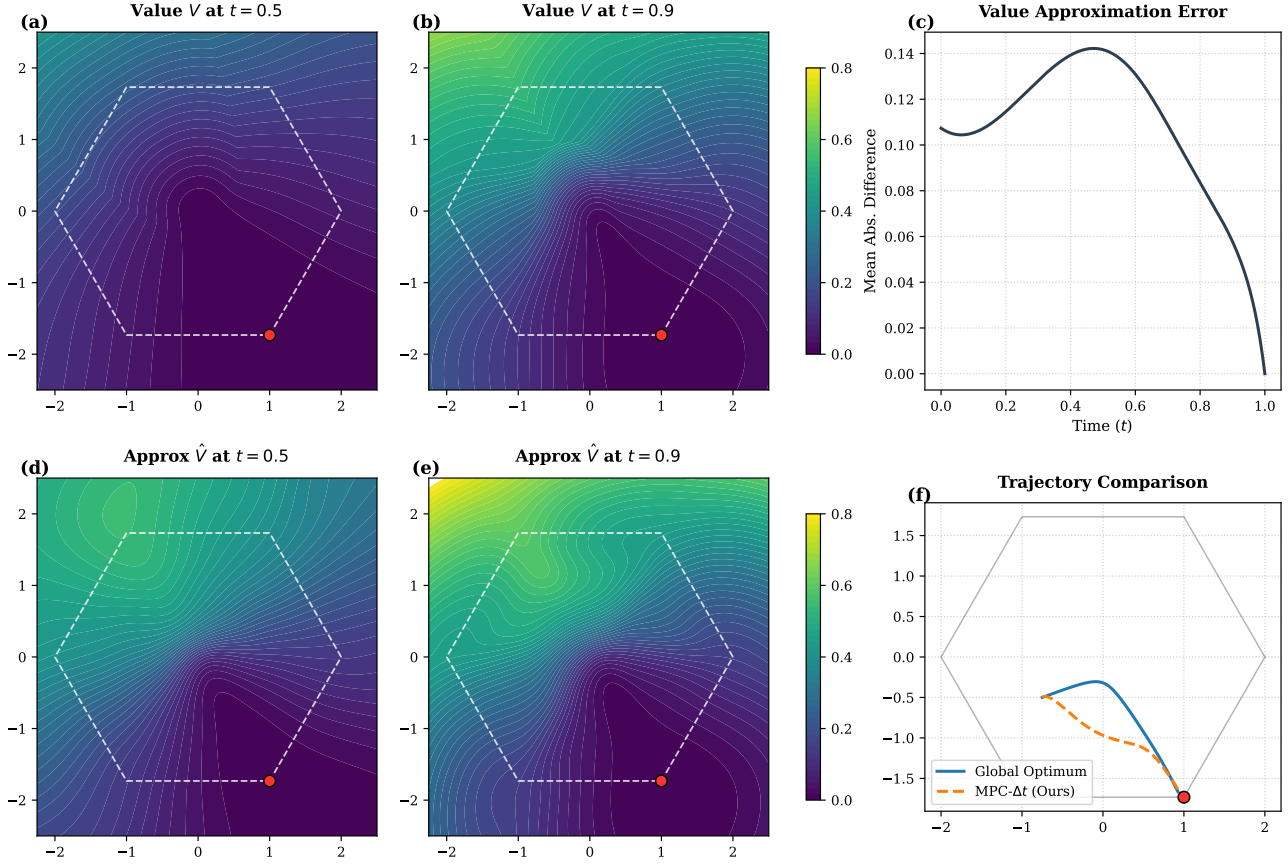


Figure C3. MPC- Δt for the 2D hexagon toy example. In (a) and (b) we show the value function (computed using the HJB) at $t = 0.5$ and $t = 0.9$. Similarly, in (d) and (e) we show the respective one-step approximation. The approximation error over time is in (c). Finally, (f) shows the global optimal control and the trajectory computed with MPC- Δt .

objective function at time t' then reduces to

$$\min_{\mathbf{u} \in \mathbb{R}^d} \left\{ \mathcal{J}_{K=1}(\mathbf{u}) = (1 - t') \|\mathbf{u}\|^2 + \frac{\lambda}{2} \|\mathbf{A}\mathbf{x}' - \mathbf{y}\|^2 \right\}, \quad \mathbf{x}' = \mathbf{x} + (1 - t')(\mathbf{v}_\theta(\mathbf{x}, t') + \mathbf{u}), \quad (17)$$

and the gradient w.r.t. the control $\mathbf{u}$ is given as

$$\nabla_{\mathbf{u}} \mathcal{J}_{K=1}(\mathbf{u}) = (1 - t') [2\mathbf{u} + \lambda \mathbf{A}^T (\mathbf{A}\mathbf{x}' - \mathbf{y})] \quad (18)$$

In particular, the second term of the gradient is orthogonal to the null space of $\mathbf{A}$. This means that in directions of the null space the gradient only depends on the quadratic regularisation term. As we initialise the control with zero, components of $\mathbf{u}$ corresponding to the null space of $\mathbf{A}$ remain zero. Consider the super-resolution tasks. Here, super-resolution is implemented as keeping only every second pixel. In Figure 4 the null space of the operator, corresponding to a pixel grid, is still visible in the MPC-RHC $K=1$ reconstruction and leads to artifacts. For these applications, the control is seemingly changing the observed pixels too fast and the flow model is not able to properly fill in the missing information.

F. Image generation with Flux.2

Style loss explicit definition We define

$$\Phi(\mathbf{z}) = \|\text{Style}(\text{Dec}(\mathbf{z})) - \text{Style}(\mathbf{x}_{\text{ref}})\|_2^2,$$

where the Style operator uses CLIP ViT-B/16 (Radford et al., 2021) image features. Let $F(\mathbf{x}) \in \mathbb{R}^{N \times d}$ denote the patch features extracted from the image encoder, and define the Gram matrix $G(\mathbf{x}) = F(\mathbf{x})^\top F(\mathbf{x})$. We then set $\text{Style}(\mathbf{x}) = G(\mathbf{x})$ and take $\mathbf{x}_{\text{ref}}$ to be the reference style image.

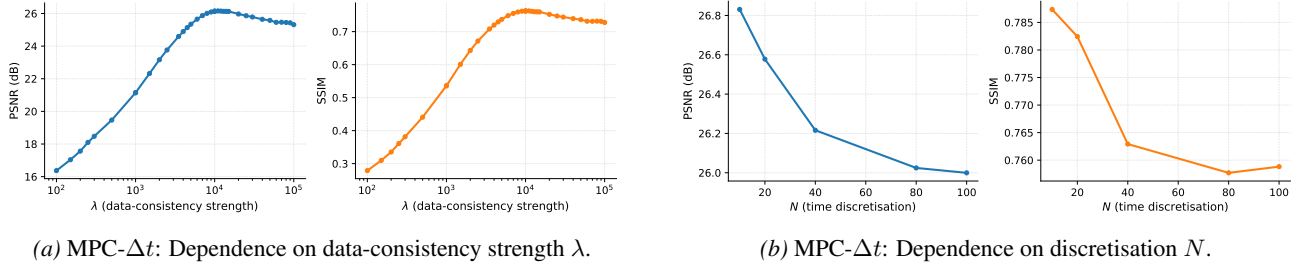


Figure D1. Effect of $\Delta t = 1/N$ and the data-consistency strength λ on the performance of MPC- Δt . We plot both the mean PSNR and SSIM over 10 images from the test set.

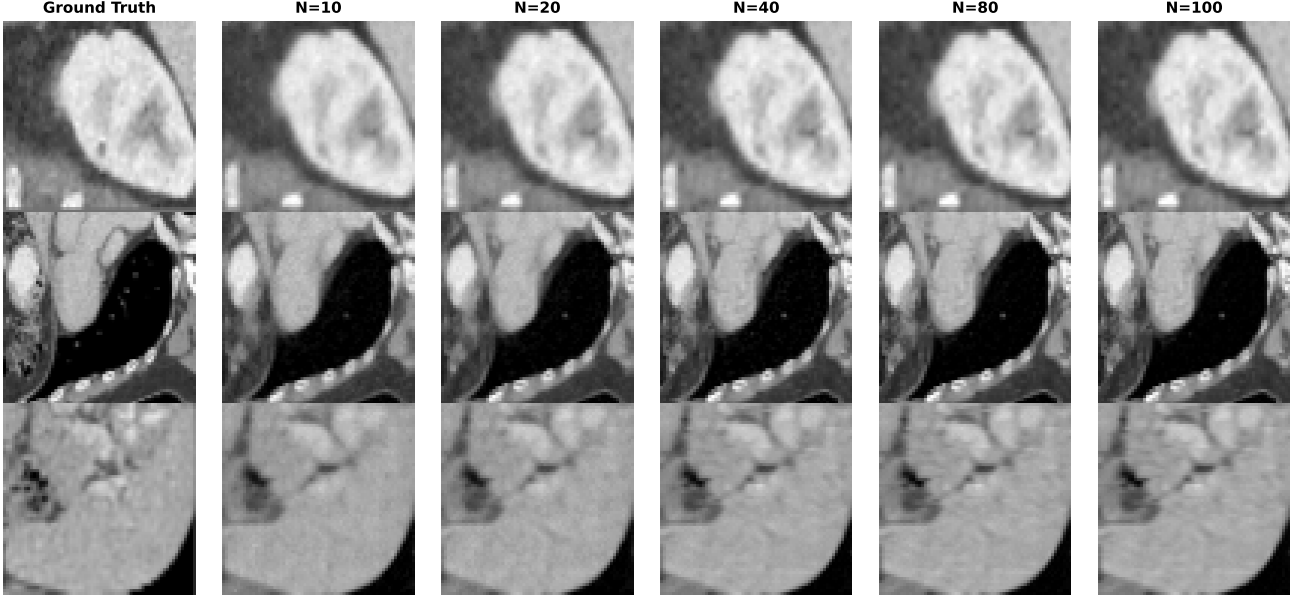


Figure D2. Example reconstruction using MPC- Δt with $\lambda = 1 \times 10^4$ with different choices of the discretisation resolution $N = 10, 20, 40, 80, 100$. For lower N , the images are slightly smoother, giving rise to a higher PSNR in Figure D1b.

Super-resolution Figure F1 shows example images for the 8x super-resolution task using MPC-Flow and FlowChef with FLUX.2.

Hyperparameter settings We used FLUX.2’s default image-generation settings: 28 denoising steps and a guidance scale of 4. For FlowChef, we implemented Algorithm 1 from Patel et al. (2024).

For both FlowChef and MPC-Flow, we fixed a budget of 20 inference steps per denoising step. In FlowChef, the strength of conditioning was controlled via the learning rate on the conditioning objective, which was varied from 1×10^{-4} to 5×10^{-1} . In MPC-Flow, rather than scaling the terminal objective directly, we rewrote the control objective

$$\int_t^1 \|\mathbf{u}(\tau)\|_2^2 d\tau + \lambda \Phi(\mathbf{x}(1))$$

into the equivalent form

$$\int_t^1 \rho \|\mathbf{u}(\tau)\|_2^2 d\tau + \Phi(\mathbf{x}(1)).$$

This reparameterisation isolates the trade-off between control regularisation and terminal quality into a single path-regularisation parameter ρ , yielding a numerically more stable tuning knob.

The choice of learning rate and ρ depended on the form and scale of the terminal cost Φ . In the style transfer experiments, Φ was defined as a perceptual loss computed through a neural network, whereas in the luminance (recoloring) experiments, Φ

Method	Image restoration tasks					
	Denoising	Deblurring	Super-res.	Rand. inpaint.	Box inpaint.	Nonl. debl.
FlowChef	(0.01,1)	(0.01,1)	(0.1,100)	(1.0,100)	(0.01,10)	(0.01,10)
FlowDPS	(0.1,100)	(0.5,75)	(0.1,100)	(0.1,100)	(0.1,100)	(0.01,100)
FlowGrad	(0.01,1.5,80)	(0.01,1.5,80)	(0.01,1.5,80)	(0.01,1.0,80)	(0.01,1.0,80)	(0.01,0.5,100)
OC-Flow	(0.01,0.5,60,0.995)	(0.01,0.5,60,0.995)	(0.01,0.5,60,0.995)	(0.01,0.5,60,0.995)	(0.01,0.5,60,0.995)	(0.01,0.5,80,0.995)
MPC- Δt	(15,20,0.1)	(7.5,20,0.1)	(7.5,20,0.1)	(0.5,20,0.1)	(5,20,0.1)	(20,20,0.1)
MPC-RHC ($K = 1$)	(0.1,20,0.1)	(0.063,20,0.1)	(1,20,0.1)	(1,20,0.1)	(0.040,20,0.1)	(0.063,20,0.1)
MPC-RHC ($K = 3$)	(0.1,20,0.05)	(0.063,20,0.05)	(0.063,20,0.05)	(1,20,0.05)	(1,20,0.05)	(0.063,20,0.05)

Table E2. Hyperparameter settings across image restoration tasks. We report (lr, N_{optim}) for FlowGrad and for FlowDPS, where N_{optim} is the number of optimization steps, (λ , lr, max-iters) for FlowGrad, (λ , lr, max-iters, weight_decay) for OC-Flow, and (λ , N_{ctrl} , lr) for MPC-based methods, where N_{ctrl} denotes the number of control optimisation steps used to solve the optimisation problem in line 5 of Algorithms 1, 2 and 3.

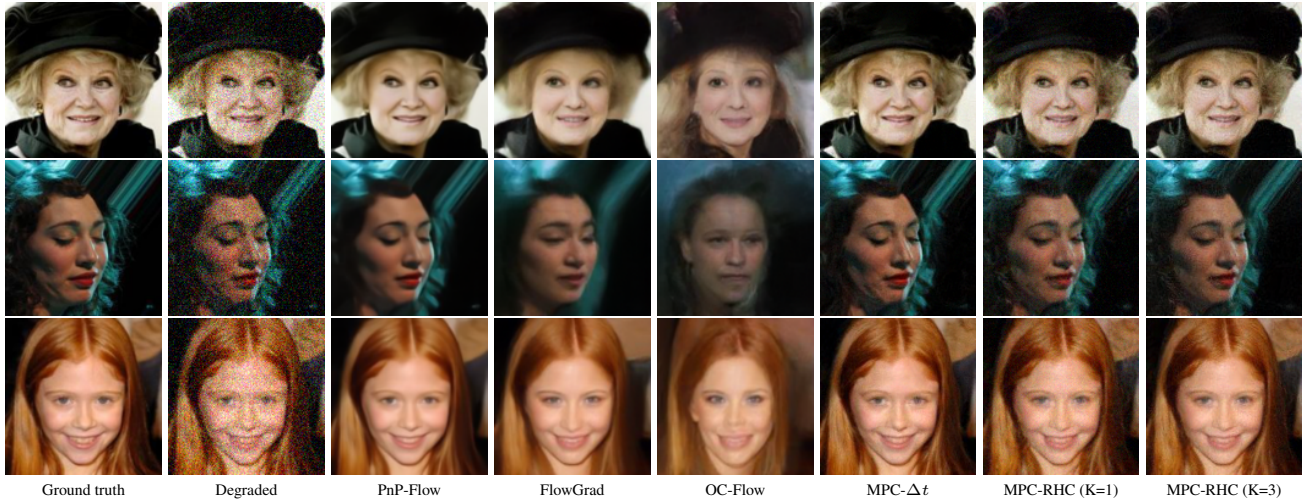


Figure E1. Qualitative results on the CelebA dataset for the denoising task with noise level $\sigma = 0.2$.

measured pixel-wise consistency between the decoded image and the target luminance image.

For style transfer, we varied ρ from 4.0 to 1024.0 and used a fixed learning rate of 0.5. For recoloring, we varied ρ from 1×10^{-8} to 5×10^{-4} and used a learning rate of 0.2.

The images in Figure 5 were generated with different prompts and reference style images for our method $\rho = 8.0$ for our method and learning rate = 0.025 for FlowChef.

Model origin We used the 4-bit quantised model Flux.2 [dev]³ obtained from the Diffusers Python package (von Platen et al., 2022). We make use of the Flux.2 [dev] model under the terms of the FLUX [dev] Non-Commercial License v2.0.

Memory usage Inference-time conditioning can be run within tight GPU memory budgets by keeping heavyweight components off-GPU when they aren’t needed. The components of interest are: text encoder (14.7 GB), latent space decoder (0.17 GB), Flux transformer (17.3 GB, quantised to 4 bits). In our setup, we move the text encoder to GPU to compute prompt embeddings, and then back to CPU. The Flux transformer and VAE are then moved to GPU for the denoising and control steps. This staged execution, combined with prompt-embedding caching and aggressive cache clearing between runs, keeps peak VRAM bounded below 24 GB even when using inference-time rewards.

Dataset construction For style transfer, we used five prompts (“a cat”, “a landscape”, “a portrait”, “a dog” and “two lizards wearing fedoras”) paired with each of nine style images to construct a set of 45 style transfer tasks that we evaluated methods on. Images were generated at 448×448 resolution.

³<https://huggingface.co/diffusers/FLUX.2-dev-bnb-4bit>



Figure E2. Qualitative results on the CelebA dataset for the image deblurring task with noise level $\sigma = 0.05$ and std of blurring Gaussian kernel set to $\sigma_b = 1.0$.

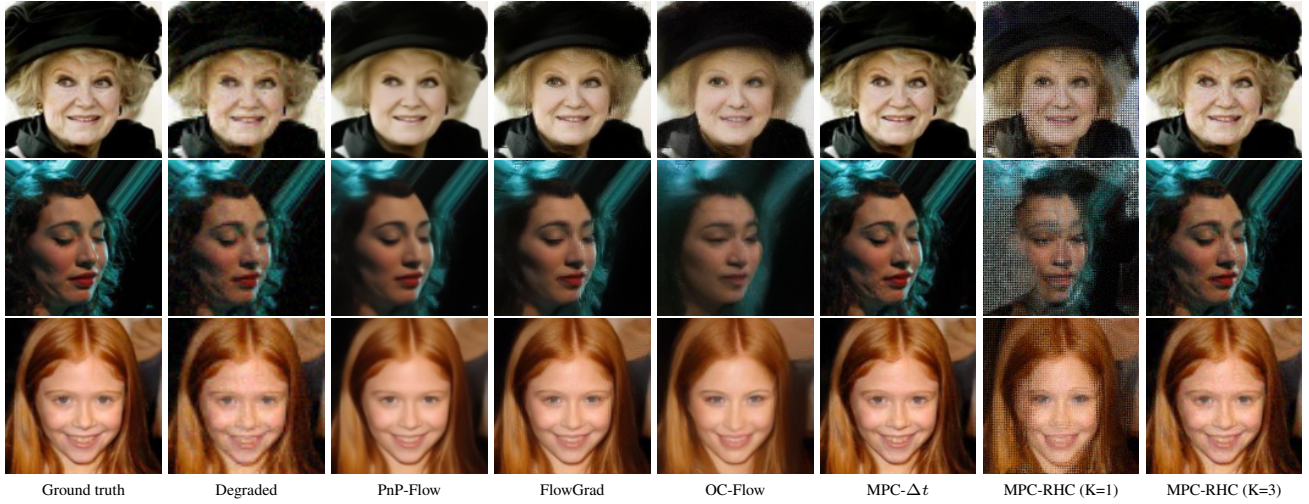


Figure E3. Qualitative results on the CelebA dataset for the image super-resolution task with noise level $\sigma = 0.05$ and $\times 2$ upscaling.

For image colouration, we used ten images sourced from Openverse⁴, which were cropped and resized to 256×256 resolution.

In Figure F2, "lisa-the-dog" by roman.schurte is marked with CC0 1.0; "High School Ring" by slgckgc is licensed under CC BY 2.0; "Piano" by Sean MacEntee is licensed under CC BY 2.0; "shoes" by kats_stock_photos is licensed under CC BY 2.0; "The Iconic Image" by Andy Morffew is licensed under CC BY 2.0. -

G. Image Restoration with Stable Diffusion 3.0

To demonstrate the flexibility of our framework across latent flow models, we qualitatively compare MPC-Flow with $K = 1$ against FlowDPS (Kim et al., 2025) and FlowChef (Patel et al., 2025), using Stable Diffusion 3.0 (SD3.0) (Esser et al., 2024). We evaluate the methods on three samples drawn from the DIV2K train set, the AFHQ train set, and the FFHQ train set, respectively, using the same examples provided in the official FlowDPS repository.⁵ Following FlowDPS, we consider three inverse problems: (i) super-resolution (SR) from average pooling with a scale factor of 12, (ii) SR from

⁴<https://openverse.org/>

⁵<https://github.com/FlowDPS-Inverse/FlowDPS>



Figure E4. Qualitative results on the CelebA dataset for the random image inpainting task with noise level $\sigma = 0.05$ and 70% random masking. OC-Flow fails to produce meaningful restorations and is therefore indicated as failed.

Algorithm 3 Single-Step RHC

input Pretrained flow model $v_\theta(\mathbf{x}, t)$, terminal loss $\Phi(\mathbf{x})$, initial state $\mathbf{x}_0$, control step size δ

```

1:  $t' \leftarrow 0, \quad \mathbf{x} \leftarrow \mathbf{x}_0$ 
2: while  $t' < 1$  do
3:   Initialise control  $\mathbf{u}$  (e.g., warm-start)
4:   solve:
5:      $\mathbf{u}^* = \arg \min_{\mathbf{u}} [(1 - t')\|\mathbf{u}\|^2 + \Phi(\mathbf{x}')]$ 
6:     with:  $\mathbf{x}' = \mathbf{x} + (1 - t')(v_\theta(\mathbf{x}, t') + \mathbf{u})$ 
7:   Apply control  $\mathbf{u}_0^*$  on interval  $[t', t' + \delta]$ 
8:   Update state:  $\mathbf{x} \leftarrow \mathbf{x} + \delta(v_\theta(\mathbf{x}, t') + \mathbf{u}^*)$ 
9:   Update time:  $t' \leftarrow t' + \delta$ 
10: end while
    
```

bicubic interpolation with a scale factor of 12, and (iii) Gaussian deblurring with a kernel size of 61 and standard deviation 3.0. The resulting qualitative comparisons are shown in Figure F3.

H. Interpretation as a Regulariser for Inverse Problems

Let us consider inverse problems with additive Gaussian noise, i.e.,

$$\mathbf{y} = \mathcal{A}(\mathbf{x}) + \epsilon.$$

In this case, we can choose the terminal loss function as $\Phi(\mathbf{x}) = \|\mathcal{A}(\mathbf{x}) - \mathbf{y}\|^2$, measuring the consistency of the reconstruction $\mathbf{x}$ to the observations $\mathbf{y}$. Let us consider the global optimal problem (5), which we re-state here as

$$\begin{aligned}
 & \min_{\mathbf{u}} \left\{ \mathcal{J}_\rho(\mathbf{u}) = \|\mathcal{A}(\mathbf{x}(1)) - \mathbf{y}\|^2 + \rho \int_0^1 \|\mathbf{u}(\tau)\|_2^2 d\tau \right\} \\
 & \text{s.t.} \quad d\mathbf{x}(t)/dt = v_\theta(\mathbf{x}(t), t) + \mathbf{u}(t), \\
 & \quad t \in [0, 1], \mathbf{x}(0) = \mathbf{x}_0,
 \end{aligned}$$

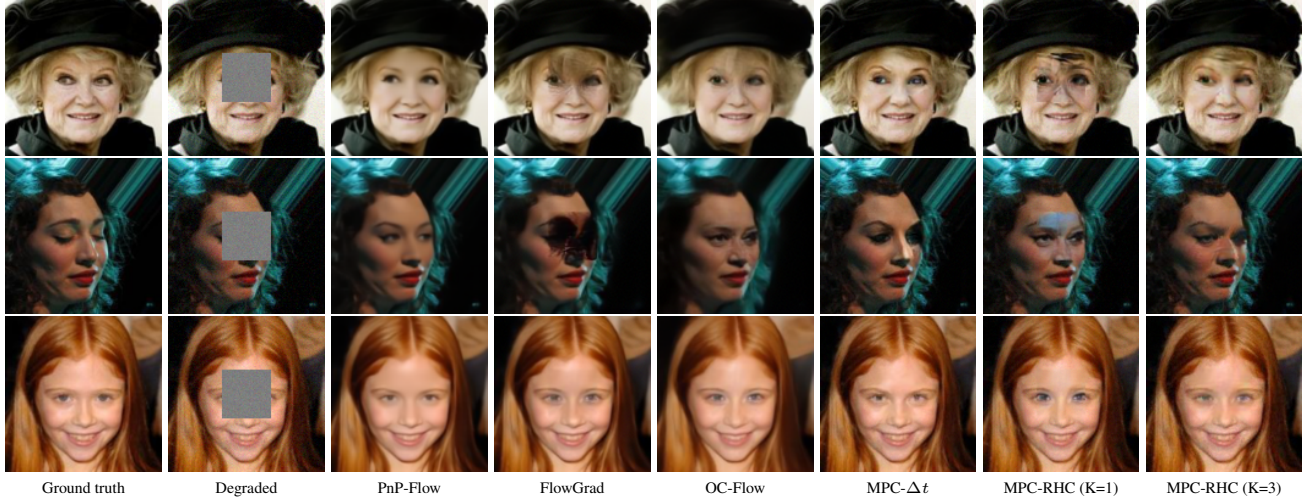


Figure E5. Qualitative results on the CelebA dataset for the box inpainting task with noise level $\sigma = 0.05$, and a central missing region of size 40×40 pixels.

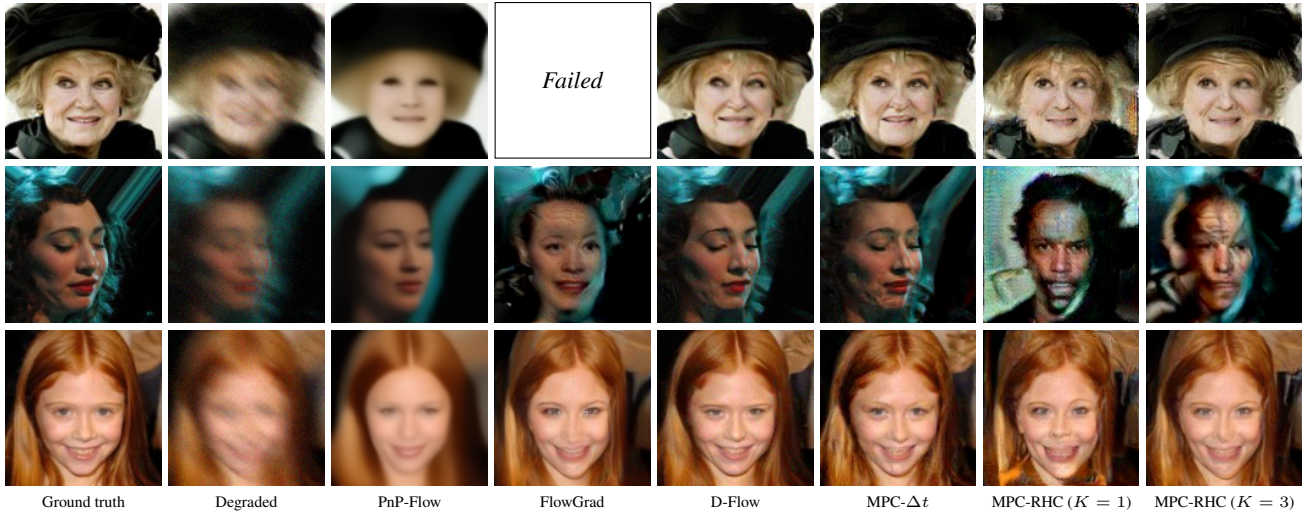


Figure E6. Qualitative results on the CelebA dataset for the nonlinear blur task with noise level $\sigma = 0.05$.

with $\rho = 1/\lambda$. We can now interpret this as a classical regularisation functional in the variational regularisation framework (Engl et al., 1996). For this, we consider the objective function

$$\arg \min_{\mathbf{x}_1} \|\mathcal{A}(\mathbf{x}_1) - \mathbf{y}\|^2 + \rho R(\mathbf{x}_1), \quad (19)$$

with R defined as the solution to a constrained optimal control problem

$$R(\mathbf{x}_1) = \inf_{\mathbf{u}} \left\{ \int_0^1 \|\mathbf{u}(\tau)\|^2 d\tau \left| \frac{d\mathbf{x}_t}{dt} = v_\theta(\mathbf{x}_t, t) + \mathbf{u}(t), \quad \mathbf{x}(0) = \mathbf{x}_0, \quad \mathbf{x}(1) = \mathbf{x}_1 \right. \right\}. \quad (20)$$

This can be interpreted as a type of dynamic prior, which penalises $\mathbf{x}_1$ if it is not *easily reachable*, i.e., there does not exist a trajectory with a small control, from $\mathbf{x}_0$. In this way, the flow model defines as distance

$$d_{v_\theta}(\mathbf{x}_1, \mathbf{x}_0) = \inf_{\mathbf{u}} \left\{ \int_0^1 \|\mathbf{u}(\tau)\|_2^2 d\tau \left| \frac{d\mathbf{x}_t}{dt} = v_\theta(\mathbf{x}_t, t) + \mathbf{u}(t), \mathbf{x}(0) = \mathbf{x}_0, \mathbf{x}(1) = \mathbf{x}_1 \right. \right\}, \quad (21)$$

which explicitly highlights the influence of the initial value $\mathbf{x}_0$. We get an interesting special case if we set the flow model to zero, i.e., $v_\theta = 0$. In this case the dynamics reduce to $\frac{d\mathbf{x}_t}{dt} = \mathbf{u}(t)$ and the optimal solution is simply a straight line from $\mathbf{x}_0$ to $\mathbf{x}_1$ and the distance reduces to $d_0(\mathbf{x}_1, \mathbf{x}_0) = \|\mathbf{x}_1 - \mathbf{x}_0\|^2$ and we obtain a typical Tikhonov regularisation functional.



Figure F1. Example images for 8x super-resolution task with FLUX.2. Proj PSNR refers to the PSNR of the low-resolution image as measured in the high-resolution space (by upsampling with the adjoint of the downsampling operator).

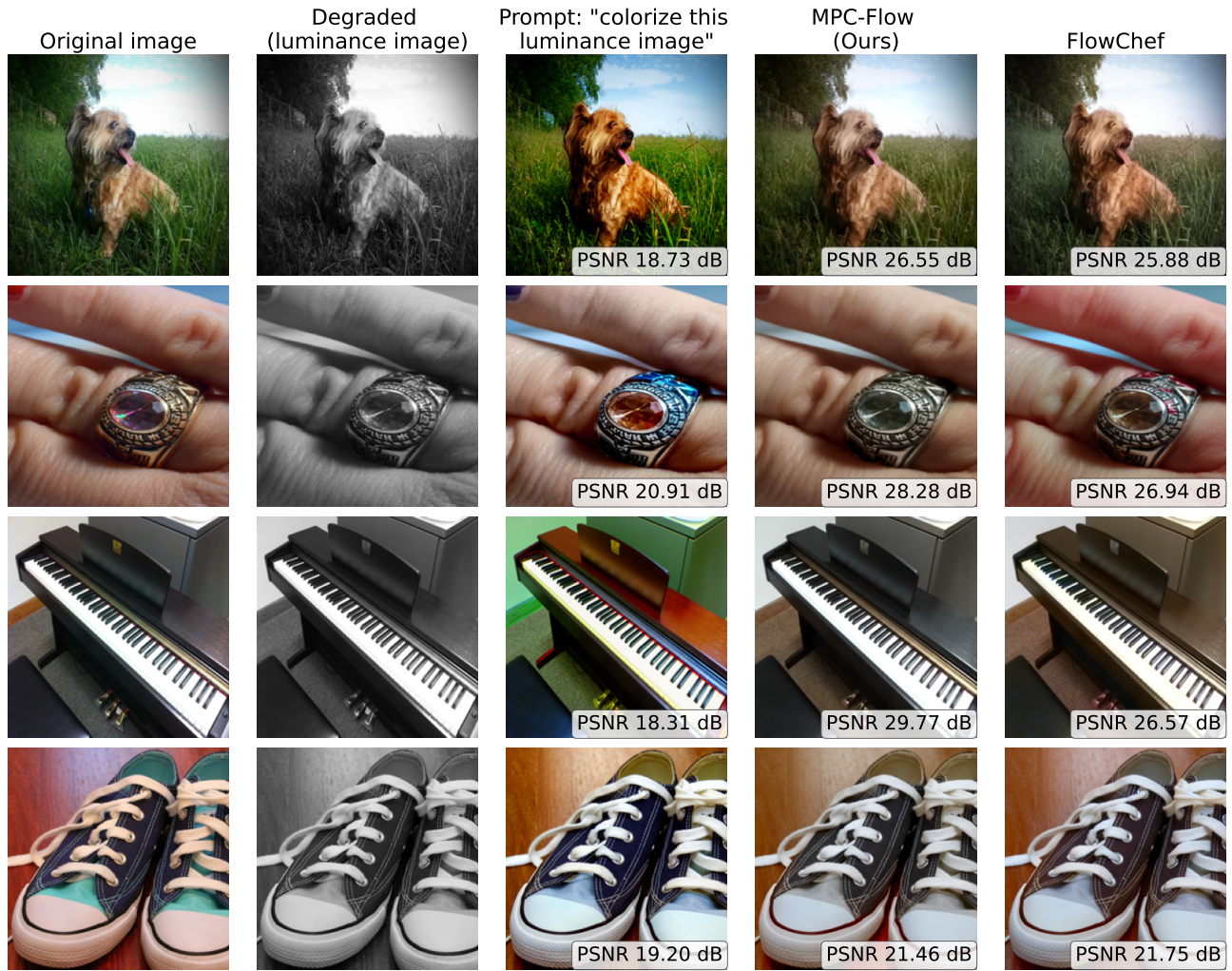


Figure F2. Further examples of image recolouration with Flux.2 and MPC-Flow.

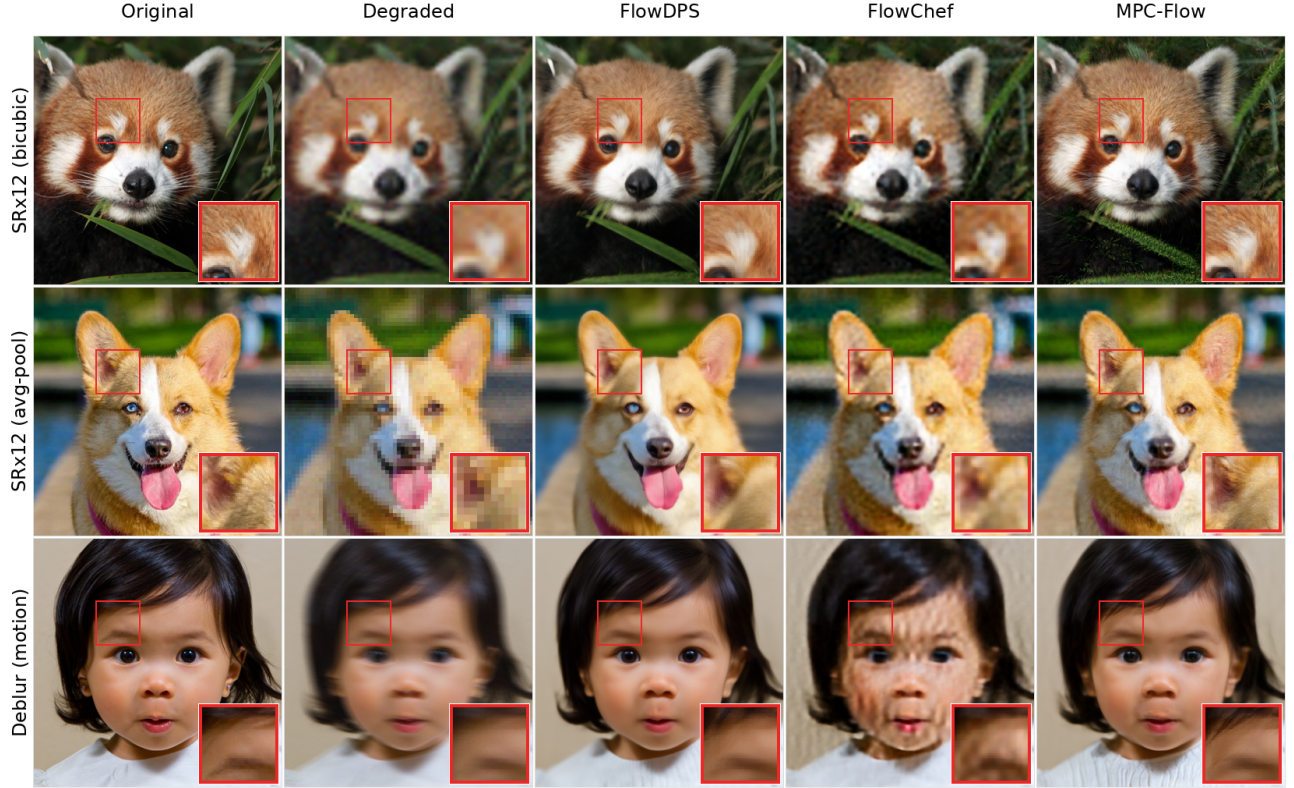


Figure F3. Qualitative comparison of MPC-Flow with $K = 1$ against FlowDPS (Kim et al., 2025) and FlowChef (Patel et al., 2025). The first, second, and third rows correspond to samples from the DIV2K train set, AFHQ train set, and FFHQ train set, respectively. Following the FlowDPS evaluation setup, the three tasks are: super-resolution from average pooling with scale factor 12, super-resolution from bicubic interpolation with scale factor 12, and Gaussian deblurring.